
모델이 잠들 때 무엇이 일어나는가?

Sleep-Time Compute, 지속학습, 장기기억을 하나의 생애주기로 보는 연구 모노그래프

2026-07-25

Contents

초록

1. 작동적 정의: sleep은 밤이 아니라 경계다
2. 시간순 계보: 이름보다 오래된 아이디어, 이름 이후 갈라진 구현
3. 생물학적 비유가 주는 것과 주지 않는 것
4. 두 축을 넘어서 기억 매질 지도
5. Sleep에서 무엇을 학습하는가: data, objective, policy
6. 용량, 망각, plasticity: 무엇이 먼저 차는가
7. Wake-sleep-장기 기억의 통합 시스템
8. 제안 scaling laws와 claim-down 규율
9. 검증 프로그램: 이름이 아니라 operator를 비교한다
10. Open questions와 검증 가능한 가설
11. 이 연구를 반증하는 결과
12. 단계별 연구 agenda와 산출물
13. 결론: 모델이 “자느” 것이 아니라 기억 생애주기가 생긴다

- 문헌 기준일: 2026-07-25
- 문서 성격: 감사된 1차 문헌과 구현을 바탕으로 한 position/review 및 검증 가능한 연구 프로그램
- 결과 경계: 이 문서는 아직 수행되지 않은 benchmark의 결과를 주장하지 않는다.
- 근거 모체: [사전 조사](#), [생물학·계산적 sleep 감사](#), [초기 wake-sleep 선행연구 감사](#), [agent-memory system 감사](#), [회사별 연구 계보 감사](#), [용량·공고화 이론 감사](#), [2026년 7월 연구 감사](#), [2026 last-mile 감사](#), [systems 선행기술 감사](#), [scientific red-team 감사](#), [training/data 방법 atlas](#), [open-question matrix](#), [scaling-law agenda](#), [system blueprint](#), [benchmark blueprint](#)

문장에 붙는 증거 표지

이 글은 세 기본 종류의 문장과 그 파생 표지를 의도적으로 구분한다.

- **[확립된 근거]** 감사된 논문 또는 공식 구현이 직접 보여 준 사실이다. 다만 그 논문의 모델, 데이터, 기간, 측정법을 벗어나 일반화하지 않는다.
- **[통합 추론]** 여러 근거를 하나의 설계 언어로 묶어 얻은 해석이다. 개별 논문의 직접 결론은 아니지만, 어떤 근거에서 나왔는지 추적할 수 있다.
- **[제안 가설]** 아직 검증해야 하는 독창적 또는 확장적 명제다. 가능한 한 반증 조건과 필요한 실험을 함께 적는다.
- **파생 표지** [확립된 회계]는 stated accounting boundary 안의 항등식, [조건부 이론]은 명시한 모형·가정 아래의 이론적 귀결이다. [제안 규율], [제안 실험], [제안 아키텍처], [제안 측정]은 모두 미평가 proposal의 하위 유형이다. [확립된 근거/필요조건]처럼 slash가 붙은 표지는 앞 표지의 증거 지위를 유지하면서 뒤에 논리적 자격이나 적용 범위를 덧붙인다. 어느 파생 표지도 별도의 더 강한 empirical evidence 등급이 아니다.
- **비주장** 정의, 실험 계약, roadmap, 제한·철회 조건은 새 empirical result나 독창성·우선권 주장이 아니다. 특히 아직 실행하지 않은 benchmark의 수치와 판정은 결과가 아니라 사전 명제다.

확립은 “모든 환경에서 참”이라는 뜻이 아니다. 예를 들어 한 논문이 두 과제 SNN에서 망각 감소를 보였다면, 확립된 것은 그 실험 안의 결과이지 LLM의 무한 생애학습 법칙이 아니다. 반대로 가설은 홍보 문구가 아니다. 어떤 관찰이 나오면 철회하거나 범위를 좁힐지를 미리 선언한 연구 계약이다.

초록

[통합 추론] 오늘날 “sleep-time compute”라는 이름 아래에는 서로 다른 세 문헌군이 겹쳐 있다. 첫째는 미래 질의가 오기 전에 알려진 context를 다시 읽고 짧고 유용한 text로 만드는 **pre-query context compilation**이다. [Letta의 Sleep-time Compute](#)가 대표적이며, 핵심 operator는 weight update가 아니라 token-space 재표현이다. 둘째는 wake 동안 쌓인 text, vector, temporal graph, procedure, trajectory를 비동기로 추출·병합·요약·검증·삭제하는 **external-memory lifecycle**이다. MemGPT, Letta의 후속 sleeper,

Mem0, Zep/Graphiti, Microsoft의 Human-Inspired Memory Architecture, Auto-Dreamer 등이 서로 다른 경계에서 이 계보를 구성한다. 셋째는 경험을 latent KV, adapter, sparse slot, expert, fast weight 또는 slow model weight로 옮기는 **latent/parametric consolidation**이다. Cartridges, Sparse Memory Finetuning, Google Research의 *Language Models Need Sleep*, 그리고 더 넓게는 continual learning의 replay·distillation·regularisation·isolation 문헌이 이 축에 놓인다.

[통합 추론] 그러므로 핵심 질문은 “AI도 사람처럼 잠을 자야 하는가?”가 아니다. 더 정확한 질문은 다음과 같다.

배포 후 들어오는 경험을 언제, 어떤 자료와 목적함수로 변환하고, 어느 기억 매질과 어느 scope에 써야 하며, 유한한 용량·간섭·오염·비용·삭제 의무 아래에서 다음 wake의 효용을 어떻게 높일 것인가?

이 질문은 배포 전 학습, wake/test-time learning, sleep-time learning을 하나의 연속된 생애주기로 다시 배열한다. 배포 전에는 shared base와 일반 policy를 만든다. Wake에서는 낮은 P99 latency를 지키며 inference, retrieval, capture, 제한적 fast update를 수행한다. Sleep에서는 immutable snapshot을 바탕으로 replay, dream-data generation, distillation, SFT/RL, external compaction, graph rollup, adapter compilation을 실행한다. 그러나 sleep의 산출물은 즉시 정답이 아니다. incumbent와 비교하는 canary, retention과 acquisition의 분리 평가, provenance와 delete fan-out, atomic publication, rollback을 통과한 **candidate persistent state**다.

[확립된 근거] 2022년 블로그에서 지목된 PLOS 연구는 학회 발표가 아니라 Golden 등 연구자의 [PLOS Computational Biology](#) 논문이다. 두 개의 상보적 foraging task를 배우는 842-neuron SNN에서 wake task와 sleep-like Poisson activation·unsupervised STDP를 교대로 적용했을 때, 순차학습으로 chance까지 떨어진 첫 과제 성능을 보존했다. 그러나 old synaptic trace가 완전히 지워진 뒤에는 회복하지 못했고, 다과제·LLM·장기 용량을 검증하지 않았다. DANN은 [SSRN manuscript](#)로 실재하지만 공개적으로 감사 가능한 근거가 abstract와 metadata에 제한되며, “zero forgetting”은 확립된 결과로 사용할 수 없다. Meta 계보에서 직접 확인되는 핵심은 sleep scheduler가 아니라 [Memory Layers at Scale](#)과 [Sparse Memory Finetuning](#)의 sparse parametric capacity와 국소 update다.

[확립된 근거] 생물학은 replay와 수면 의존적 공고화가 실재하며 일부 cue-rhythm 조작이 이후 기억을 바꾼다는 강한 근거를 준다. 하지만 NREM=정확 복사, REM=창의적 생성 또는 pruning이라는 일대일 대응은 확립된 neuroscience가 아니다. [Perturbed and Adversarial Dreaming](#), [Singh-Norman-Schapiro의 PNAS 모델](#), Google Sleep은 서로 다른 방식으로 NREM/REM 유사 역할을 공학적으로 배치했다. 이들은 유용한 ablation design이지 뇌의 알고리즘을 입증한 것이 아니다.

[통합 추론] 외부기억과 모수기억은 양자택일이 아니라 서로 다른 위험과 비용을 지닌 destination이다. 가장 보수적인 생애주기는 exact raw event를 provenance와 함께 붙잡고, 반복되는 구조를 text/vector/temporal graph로 정리하며, 높은 재사용과 안정성이 검증된 것만 latent KV나 user-local adapter로 compile하고, 가장 높은 재사용·낮은 volatility·낮은 삭제 위험을 가진 구조만 shared slow weight로 승격하는 사다리다. 이때

external memory는 단순한 보조 데이터베이스가 아니라 **증거 축적층**, latent·adapter는 **serving cache**, shared weight는 **고비용·고재사용 compiled knowledge**가 된다. 모든 승격은 역방향 lineage와 demotion·rebuild·rollback 경로를 남겨야 한다.

[통합 추론] “계속 배우면 기억용량을 넘는가?”에 대한 답은 그렇다. 다만 가장 먼저 차는 자원이 하나가 아니다. 물리 저장 공간, 표현·보존 용량, 필요한 기억을 실제로 찾는 addressability, 새 것을 배울 수 있는 plasticity, provenance·consent·delete·rollback을 처리하는 governance, sleep service backlog, accelerator hot residency가 서로 다른 시점에 binding constraint가 된다. Generative replay는 memory를 없애지 않고 generator·teacher·seed·compute·오류 부채로 옮긴다. Adapter isolation은 interference를 module count·routing·HBM residency 문제로 바꾼다. Bounded live prompt는 전체 archive와 version graph가 bounded하다는 뜻이 아니다. MEMENTO가 보여 준 것처럼 visible summary가 같아도 source-conditioned KV와 restart KV의 효용이 다를 수 있으므로, implicit state도 기억 회계에 들어가야 한다.

[제안 가설] 본 모노그래프는 sleep-time scaling을 단일 exponent가 아니라 여러 crossover와 capacity knee의 묶음으로 제안한다. 예상 재사용이 compilation cost를 상쇄하는 reuse break-even, staleness harm과 실행비가 만드는 cadence optimum, arrival와 service가 만드는 queue boundary, admission과 physical reclamation이 만드는 byte-flow equilibrium, retrieval competition과 plasticity가 만드는 usable-capacity knee, module isolation과 router·merge·residency tax가 만나는 parametric-routing knee, temporal clock 수와 queue·abstraction debt가 만나는 clock-count crossover가 그 예다. 이 관계들은 보편 법칙으로 선언되지 않는다. frozen workload, resource tuple, measurement unit, held-out scale에서 예측에 성공할 때만 “regime-specific scaling law”라고 부르고, 아니면 empirical curve, null, narrowed claim으로 내려야 한다.

[제안 가설] 가장 중요한 연구 산출물은 새 summarizer 하나가 아니다. 필요한 것은 같은 wake stream을 external, latent, user-parametric, static mixture, past-only hybrid router에 배분하고, 동일한 information cutoff와 resource envelope에서 128-cycle lifetime utility, retention, fresh-normalized acquisition, deletion, rollback, latency, bytes, FLOPs, data movement를 함께 측정하는 transactional cross-substrate benchmark다. Positive result만큼 null result도 가치가 있다. Hybrid가 강한 baseline을 이기지 못하더라도, 어떤 capacity가 먼저 binding되는지, 어떤 dream mixture가 희귀 기억을 파괴하는지, 어떤 claim이 exact-repeat cache 효과에 불과한지를 재현 가능하게 밝히면 이 분야의 주장 수준을 실질적으로 높인다.

1. 작동적 정의: sleep은 밤이 아니라 경계다

1.1 배포 전, wake, sleep을 구분하는 최소 언어

[통합 추론] 앞으로의 모델 학습을 시간축으로 나누면 세 층이 자연스럽다.

1. **배포 전 pre-/post-training**은 대규모 corpus와 비교적 느슨한 latency 아래 base checkpoint, instruction-following policy, safety policy, memory writer 또는 router의 초기값을 만든다.
2. **wake/test-time learning**은 실제 사용자·환경 stream을 받으면서 응답, retrieval, capture, current-session state, 제한적 online update를 수행한다. 무엇이든 현재 action path의 결과에 필요하거나 같은 task를 다시 시도하기 전에 갱신된다면 wake 쪽이다.
3. **sleep-time learning**은 완료된 wake 경험의 immutable snapshot을 읽고, 현재 action의 결과를 바꾸지 않으며, 만들어진 persistent state가 이후 wake에서 처음 사용되도록 변환한다.

이 구분은 wall-clock 밤, GPU idle, 함수 이름 async, 저자의 생물학적 비유에 의존하지 않는다. 서버가 24시간 요청을 받더라도 snapshot과 version을 사용하면 wake와 sleep은 물리적으로 겹칠 수 있다. 반대로 새 응답을 만들기 전에 await한 background coroutine은 이름이 sleep이어도 작동적으로 wake다.

[통합 추론] 감사된 방법을 strict sleep으로 부르기 위한 세 조건은 다음과 같다.

- C1: 완료되었거나 축적된 wake experience를 입력으로 받는다.
- C2: 변환이 그 experience 뒤, 사용자-visible action path 밖에서 또는 immutable snapshot 위에서 실행된다.
- C3: 산출물이 지속되고 이후 wake에서 재사용된다.

SIESTA의 periodic latent replay, PCMC의 sleep encoder retraining·re-indexing, Auto-Dreamer의 offline external-memory replacement, Microsoft HIMA의 scheduled tier consolidation, Google Sleep의 fast-to-slow expert transfer는 이 정의에 가깝다. 반면 [Test-Time Training](#), TTT layer, Memento 2, Memento-Skills 논문 본체, MEMENTO의 within-query compaction은 지속 state를 만들 수 있더라도 현재 query/task의 실행 경계 안에서 동작하므로 비교해야 할 wake learner다.

1.2 sleep operator의 완전한 기술

[통합 추론] “이 방법은 sleep을 한다”는 서술만으로는 재현도 비교도 불가능하다. 적어도 다음 다섯 질문에 답해야 한다.

wake input	무엇을 경험으로 수집했는가?
trigger	어느 조건이 job을 열었는가?
operator	retain/merge/abstract/rewrite/replay/distill/SFT/RL/grow/prune 중 무엇을 했는가?
destination	text/vector/graph/KV/adaptor/fast weight/slow weight 중 어디에 썼는가?
capacity	무엇을 보존·검증·버리며, 가득 찼을 때 무엇을 하는가?

여기에 scope를 추가해야 한다. 같은 LoRA라도 session-local, user-local, cohort, shared scope는 privacy와 interference의 blast radius가 전혀 다르다. 같은 graph도 한 사용자의 temporal history인지 여러 tenant가 공유하는 semantic layer인지에 따라 publication과 deletion 계약이 달라진다.

1.3 수학적 작동 정의

Wake가 사건 stream $e_{1:t}$ 와 임시 상태 h_t 를 만들고, persistent memory를

$$M_t = \{M_{\text{raw}}, M_{\text{text}}, M_{\text{vector}}, M_{\text{graph}}, M_{\text{KV}}, M_{\text{adapter}}, M_{\text{fast}}, M_{\text{slow}}\}$$

라고 하자. Sleep policy S_ϕ 는 resource budget B_s , governance constraint R , incumbent version v 아래

$$\widehat{M}_{v+1} = S_\phi(e_{1:t}, M_v; B_s, R)$$

라는 candidate를 만든다. 다음 wake의 answer policy는 live budget B_{live} 아래

$$a = A_\theta(q, \text{retrieve}(M_{v+1}, q); B_{\text{live}})$$

를 계산한다. 중요한 기호는 hat이다. Sleep output은 검증과 atomic publication 이전에는 serving state가 아니다.

[통합 추론] 최적화 대상은 answer accuracy 하나가 아니라 대략 다음 vector다.

future-task loss
live latency와 token
sleep FLOPs·energy
persistent·peak bytes
tiers 사이 bytes moved
forgetting·retrieval competition·plasticity loss
stale/conflicting/false-memory loss
privacy·poison·delete·rollback risk

가격과 선호 가중치가 정당화되면 scalar objective로 바꿀 수 있다. 그렇지 않으면 서로 다른 단위를 억지로 더하지 않고 constrained Pareto frontier로 남겨야 한다. 이 원칙은 “offline cost를 빼고 online token만 비교”하거나, 반대로 baseline의 retrieval 비용은 세면서 candidate의 teacher·index rebuild·rollback 비용은 빼는 비대칭을 막는다.

1.4 scheduling label과 computational phase를 가르는 실험

[제안 가설] Delayed execution 자체가 의미 있는 computational phase라면, 동일한 사건

·operator·optimizer step·destination·future query를 주고 즉시 wake update와 snapshot-based deferred update를 비교했을 때 적어도 하나의 축에서 이득이 있어야 한다. 후보 이득은 wake P99 격리, 더 큰 batch, 더 넓은 cross-session evidence, 더 강한 verifier, safer rollback이다.

반증 조건. Snapshot·queue·promotion 비용까지 포함한 뒤 deferred arm이 utility, cost, latency, safety, amortization 어느 축에서도 compute-matched immediate arm보다 낮지 않다면, 그 regime에서 “sleep”은 독립적인 학습 원리라기보다 scheduling label이다. 이 반증은 모든 sleep 가능성을 부정하지 않고, 범위를 해당

operator와 workload로 좁힌다.

2. 시간순 계보: 이름보다 오래된 아이디어, 이름 이후 갈라진 구현

2.1 1983–2007: 꿈, replay, fast/slow system의 이론적 뿌리

[확립된 근거] [Crick와 Mitchison의 1983년 *The Function of Dream Sleep*](#)은 REM의 quasi-random activation이 기생적 attractor를 활성화하고 anti-Hebbian하게 약화한다는 reverse-learning 가설을 제안했다. 데이터셋, optimizer, 동물·사람 실험은 없었고 저자도 추측적 성격을 인정했다. 이 논문은 pruning과 unlearning의 역사적 출발점이지 “REM이 불필요한 기억을 삭제한다”는 확립된 생물학이 아니다.

[확립된 근거] [Buzsáki의 1989년 two-stage memory trace 모델](#), [Wilson과 McNaughton의 1994년 hippocampal ensemble reactivation](#), [Skaggs와 McNaughton의 1996년 압축 sequence replay](#)는 wake에서 형성된 경험 관련 활동이 slow-wave sleep에서 다시 나타날 수 있음을 보였다. [Complementary Learning Systems](#)는 빠른 episodic acquisition과 느린 distributed structure learning을 분리하고, recent memories를 기존 경험과 interleave해야 interference를 줄일 수 있다고 논증했다. Fast store는 영구 archive라기보다 slow learner의 teacher다.

[확립된 근거] 같은 1995년 [Robins의 pseudorehearsal](#)은 old raw example 없이 random input에 old network의 output을 붙여 새 data와 interleave했다. 이는 생성 replay의 직접적인 공학적 선조다. 하지만 기억이 사라진 것은 아니다. Old network의 함수가 teacher로 남는다. “raw data free”는 “past information free”가 아니다.

[확립된 근거] [Tononi와 Cirelli의 synaptic homeostasis hypothesis](#)는 wake의 평균적 potentiation을 sleep에서 선택적·활동 의존적으로 renormalize해 에너지, 공간, signal-to-noise, 다음 날 plasticity를 회복한다는 관점을 열었다. 후속 근거가 있지만 보편적 global downscaling 법칙은 아니며, REM pruning과 동일한 이론도 아니다. 이 계보가 주는 공학적 질문은 “무엇을 더 저장할까?”만이 아니라 “어떤 상태를 낮추거나 recycle해야 다음 학습이 가능한가?”다.

2.2 2005–2018: bounded state, replay, isolation, 명시적 down-time

[확립된 근거] Fusi–Drew–Abbott의 cascade synapse, Fusi–Abbott의 bounded-synapse 한계, Lahiri–Ganguli의 finite-state frontier, Benna–Fusi의 bidirectional fast/slow chain은 제한된 mutable state가 지속적 update 아래 overwrite된다는 점과 여러 time scale이 initial strength와 lifetime을 교환한다는 점을 보였다. 이 결과들은 idealized random memory와 ideal readout에 한정되므로 transformer의 factual capacity exponent로 옮길 수 없다. 다만 slow auxiliary state, precision, replay channel도 공짜가 아니며 총 memory accounting에 들어가야 한다는 원칙은 강하다.

[통합 추론] 2016–2018년 continual-learning 계보는 압력을 다른 자원으로 옮기는 네 방식으로 갈라졌다. EWC와 Synaptic Intelligence는 중요한 parameter direction의 이동을 억제했다. GEM은 episodic examples와 gradient constraints를 사용했다. Deep Generative Replay는 generator와 old solver가 만든 pseudo-experience를 새 data와 섞었다. Progressive Networks, PackNet, HAT는 old state를 freeze하거나 mask하고 새 free capacity를 소비했다. 어느 것도 용량을 없애지 않았다. Regularisation은 plastic subspace를, replay는 buffer·generator·compute를, isolation은 free slot과 routing을 소비한다.

[확립된 근거] 2017년 DGDMN은 작은 task-specific short-term modules가 가득 차면 down-time에 generated sample을 long-term module로 보내는 명시적 wake/sleep 구조를 제시했다. [FearNet](#)은 10 study session마다 recent exemplars와 Gaussian pseudo-examples를 섞어 mPFC network를 finetune한 뒤 hippocampal store를 비웠다. [Progress & Compress](#)는 active column이 새 task를 배우고 task boundary에서 distillation과 online EWC로 fixed knowledge base에 압축했다. 오늘의 LLM sleep이라는 이름보다 오래전에 down-time transfer, replay, distillation, fixed-capacity compression이 존재했다.

2.3 2019–2022: 생물학적 역할을 계산 objective로 분해

[확립된 근거] 2019년 Fachechi 등의 [Dreaming Neural Networks](#)는 Hopfield random-pattern memory에서 spurious mixture attractor를 약화하고 projection-like coupling으로 가는 offline dynamics를 분석했다. 표준 Hebbian Hopfield의 낮은 critical load를 크게 높이는 이론 결과가 있지만, 실제 semantic memory나 deep continual learning이 아니라 random associative-memory toy다. 이를 LLM의 memory capacity로 직접 인용하면 안 된다.

[확립된 근거] González 등의 [Can sleep protect memories from catastrophic forgetting?](#)는 biophysical thalamocortical model에서 NREM-like spontaneous replay가 interference 뒤 남은 sequence trace를 다시 접근 가능하게 만들 수 있음을 보였다. 하지만 새 task 학습을 더 오래 해 old trace를 완전히 파괴하면 sleep도 복원하지 못했다. Krishnan 등의 ANN→SNN sleep과 Tadros 등의 sleep replay consolidation도 과거 data의 mean activation statistic과 residual weight를 사용했다. 2022년 Nature Communications 결과에서 intrinsic replay만으로 joint training에 크게 못 미쳤고, 0.75% real rehearsal를 더했을 때 큰 폭으로 개선됐다. 작은 canonical anchor의 가치가 여기서 드러난다.

[확립된 근거] [Perturbed and Adversarial Dreaming](#)은 episodic latent를 저장하고 NREM-like phase에서 occlusion reconstruction, REM-like phase에서 latent mixture·noise·adversarial separation을 수행했다. 이것은 conservative replay와 exploratory generation에 서로 다른 objective를 준 직접 precedent다. 그러나 CIFAR-10/SVHN representation learning이며 lifetime continual learning, deletion, governance를 다루지 않았다.

[확립된 근거] [Singh, Norman, Schapiro의 PNAS 2022 모델](#)은 random seed와 short-term synaptic depression으로 attractor를 자율 순회하게 만들었다. NREM-like phase에서는 hippocampus와 neocortex가 결합해 최근 trace를 reinstatement하고, REM-like phase에서는 hippocampal influence를 끊은 neocortex가 remote attractor를 재생했다. Replay trial 자체를 내부 dynamics가 만든다는 점은 중요하지만 sleep 시작, stage 길이와 순서는 실험자가 고정했으며 100-initialization synthetic category simulation이다.

[확립된 근거] 2022년 Golden 등의 [PLOS 논문](#)은 블로그의 핵심 단서를 정확히 해명한다. Task 2만 순차학습하면 Task 1은 0.52 ± 0.02 , Task 2는 0.69 ± 0.03 이었다. Task 2와 sleep을 교대로 하면 각각 0.70 ± 0.03 , 0.68 ± 0.05 였다. 외부 input을 끄고 training 중 평균 firing rate에 기반한 Poisson stimulation과 unsupervised STDP를 사용했다. 그러나 두 task, 10개 simulated network, residual trace라는 범위다. PCA·kernel PCA·SVM 분석은 사후 representation geometry이지 manifold optimality theorem이 아니다.

2.4 2023–2024: 외부기억과 context→state compilation

[확립된 근거] [SIESTA](#)는 wake에서 frozen encoder의 compressed latent와 class mean을 축적하고, periodic sleep에서 balanced replay로 상위 network를 backprop했다. Bounded latent buffer와 명시적 sleep을 결합했지만 supervised vision, frozen lower feature, 2.02 GB buffer라는 경계가 있다. PCMC는 raw image patch를 STM/LTM centroid와 함께 보존하고 periodic sleep에서 encoder를 contrastive retrain한 뒤 centroids를 재임베딩하고 prune했다. 엄격한 offline consolidation이지만 300-epoch sleep과 raw patch storage를 요구하며 lifetime cap과 governance는 없다.

[확립된 근거] [MemGPT](#)는 bounded in-context working memory, recall DB, archival DB를 연결하고 pressure warning과 recursive summary를 사용했다. 원 논문의 agent는 현재 대화·tool·memory management를 같은 online loop에서 수행했다. 별도의 sleep-trained consolidator가 아니다. 이 MemGPT→Letta 제품 계보에서는 2025년 Letta 제품이 primary agent와 sleeper를 분리하면서 명시적 asynchronous external-memory architecture가 나타났다. 이는 전체 분야에서 Letta가 최초라는 우선권 주장이 아니다.

[확립된 근거] 2024년 [MEMORYLLM](#)은 layer별 latent-token pool에 forward-only로 쓰고 old token을 random drop하는 parametric-like memory를 제안했다. 후속 M+는 버려질 hidden token을 CPU long-term pool로 옮기고 retriever가 GPU로 불러오는 hot/cold 구조를 만들었다. [Generative Adapter](#)는 context를 읽은 hypernetwork가 배포 시 별도 gradient descent 없이 LoRA를 생성했다. 이는 external context와 parameter delta 사이의 중요한 중간점이다. 다만 hypernetwork 자체의 학습은 gradient 기반이다.

[확립된 근거] [Memory Layers at Scale](#)은 sparse trainable key-value layer를 128B memory parameters까지 확장해 backbone compute와 sparse factual capacity를 분리했다. 이 구조는 2025년 Sparse Memory Finetuning의 substrate가 된다. 동시에 Allen-Zhu와 Li의 [Knowledge Capacity Scaling Laws](#)는 controlled factual tuples에서 약 2 knowledge bits/parameter라는 조건부 측정을 제시했다. 이 수치는 자연어 사실, sequential sleep, 모든 architecture에 통용되는 상수가 아니다.

2.5 2025: sleep-time이라는 이름과 세 갈래의 분화

[확립된 근거] [Letta의 Sleep-time Compute](#)는 query 전에 context c 를 $S(c) \rightarrow c'$ 로 바꾸고, 이후 여러 query가 c' 를 재사용하는 구조를 실험했다. Stateful GSM-Symbolic와 Stateful AIME에서 유사한 accuracy에 필요한 live token을 줄였고 sleep compute scaling에 따른 accuracy gain을 보고했으며, 같은 context를

여러 query에 재사용할 때 평균 query cost가 낮아졌다. 직접 평가는 modified benchmark와 SWE-Features의 modified-file-set F1이며 production lifelong memory나 test-suite pass rate가 아니다. 경제적 이득은 미래 query의 예측 가능성과 reuse에 조건부다.

[확립된 근거] Letta의 공식 sleep agent는 primary가 recall/archival memory를 검색해 응답하고 sleeper가 shared core-memory block을 비동기 편집하는 역할 분리를 구현했다. 이 제품 evidence는 논문의 token-space benchmark와 구분해야 한다. MemGPT 2023, Letta paper 2025, Letta product sleeper를 한 버전처럼 인용하면 online memory management와 async consolidation의 차이를 지우게 된다.

[확립된 근거] [Mem0](#)는 message pair와 recent history에서 후보 기억을 뽑고 유사 기억과 비교해 ADD/UPDATE/DELETE/NOOP를 선택한다. 2025 논문과 OSS의 synchronous add path는 mostly wake/current-call 경계이고, periodic conversation-summary refresh는 background 작업이지만 hard lifetime cap이나 learned offline consolidator를 제시하지 않는다. Managed Platform V3는 pending event를 반환해 queue를 비동기 처리하지만, 감사된 contemporary core path는 paper의 update/delete operator가 아니라 one-pass ADD-only다. [Zep/Graphiti](#)는 episode, entity/fact, community summary의 temporal graph를 만들고 bi-temporal validity와 contradiction invalidation을 사용한다. Graphiti OSS는 ingestion이 동기적이고 community rebuild scheduler를 소유하지 않아 strict lifecycle 판정이 PARTIAL이다. 반면 Zep Cloud는 graph construction 전에 반환하고 completion event를 제공하므로 감사된 product configuration에서 asynchronous later-read PASS다. Managed Zep Observations는 proprietary background synthesis이므로 2025 paper와 공개 OSS core에서 분리해야 한다.

[확립된 근거] [Cartridges](#)는 source corpus에서 synthetic conversation과 QA를 만들고 full-context teacher의 동작을 compact recurrent/KV state로 distill했다. [SEAL](#)은 model이 self-edit, synthetic data, augmentation, hyperparameter 지시를 만들고 SFT 뒤 downstream reward로 self-edit policy를 학습했다. 첫째는 context→latent compilation, 둘째는 generated data→weight update다. 둘 다 미래 sleep system의 operator가 될 수 있지만, SEAL 자체가 recurring delayed lifecycle을 제공하는 것은 아니다.

[확립된 근거] Spens, Burgess, Behrens의 [offline-processing RL 연구](#)는 recurrent PPO meta-controller가 toy task별 action set에서 replay, world-model update, generated learning, no-op 등을 선택하도록 했다. Learned sleep action과 replay valuation은 LLM agent 이전에 존재했다. 다만 image, maze, relational task가 같은 action space와 value mechanism을 공유하지 않으며 production scheduler, delete, rollback은 없다.

[확립된 근거] Meta의 [Sparse Memory Finetuning](#)은 새 batch에 특이적으로 활성화된 sparse memory row를 TF-IDF 방식으로 고르고 그 일부만 update해 dense FT/LoRA보다 old capability 손실을 줄이는 방향을 보였다. 이는 블로그가 말한 Meta 연구에 가장 가까운 direct match다. 하지만 ICLR submission은 desk rejected되었으므로 preprint로 취급해야 하며, async sleep scheduler나 NREM/REM pipeline을 구현한 논문은 아니다.

[확립된 근거] DANN은 실재한다. 그러나 [SSRN record](#)에서 확인 가능한 것은 20쪽 단독저자 manuscript의 metadata와 abstract이며, architecture, dataset, metric, seed, uncertainty, ablation, code를 독립적으로 감사할 수 없었다. “최대 60% forgetting 감소”와 제목의 “Zero Forgetting”은 낮은 증거 등급의 방향성 주장으로만 남는다.

2.6 2026: fast→slow parameter, learned external sleep, capacity 반례

[확립된 근거] Google Research의 [Language Models Need Sleep](#)은 Continuum Memory System의 fast state를 새 slow MoE expert로 generalised knowledge distillation하고 RL imitation을 사용한 뒤 fast expert를 reset하는 Knowledge Seeding과, synthetic question을 만들고 LoRA·ReSTEM으로 학습하는 Dreaming을 분리했다. 명시적 parametric sleep의 가장 가까운 audited example이다. 그러나 공개 acceptance를 확인할 수 없고, prospective expert pool은 유한하며 수백 cycle 뒤 exhaustion·merge·prune·growth 정책은 검증하지 않았다.

[확립된 근거] [Auto-Dreamer](#)는 provenance-linked trajectories와 bounded working region을 읽고 fresh compact replacement memory를 만드는 external consolidator를 GRPO로 학습했다. End-to-end later agent performance가 reward다. 이는 prompt-only summarizer보다 강한 learned external sleep precedent지만 active bank의 경험적 compactness가 global lifetime cap을 뜻하지 않으며 source trajectory archive의 비용이 남는다.

[확립된 근거] Microsoft의 [Human-Inspired Memory Architecture](#)는 hot event cache, episodic vector store, semantic knowledge graph 사이에서 기본 6시간 주기의 deduplication, forgetting, maturation, reconsolidation을 명시했다. VSCode issue/event 실험의 dedup-based consolidation은 retention precision과 store reduction을 보고했지만 cadence optimum, maturation, reconsolidation 전체를 검증한 것은 아니다. Microsoft의 PlugMem과 MemMA는 각각 completed-task, completed-session 경계에서 external graph transform과 probe-guided repair를 보여 주고, RHO는 completed trajectories를 persistent instruction·skill·tool harness로 바꾼다. 이들은 strict experimental boundary를 제공하지만 autonomous production sleep service는 아니다.

[확립된 근거] [Facts in Weights](#)는 매 20 write마다 누적 fact를 fresh model에 redistill하는 periodic sleep-like arm을 포함했지만 100-write retention을 no-sleep보다 개선하지 못했고, canonical external copy의 필요를 부각했다. [Useful Memories Become Faulty](#)는 반복 LLM rewrite의 utility가 오르다가 다시 내려가며 forced consolidation보다 raw episodic retention이 강할 수 있음을 보였다. “더 많이 consolidate할수록 더 좋은 memory”라는 단조 가정을 직접 흔드는 결과다.

[확립된 근거] [Surprise-Gated Memory](#)는 frozen encoder 위에서 prediction-error residual로 high-surprise embedding을 episodic buffer에 넣고 periodic full replay로 slow readout을 학습했다. Surprise gate는 한 buffer를 절반으로 줄였지만 우수성을 입증할 seed·uncertainty가 없었다. 더 중요한 반례는 최근 다섯 task만 replay한 arm이 no-replay보다 훨씬 나뉘었다는 점이다. Fixed recent window는 bounded compute를 제공해도 old-support geometry를 파괴할 수 있다. Full-history replay는 retention을 지키지만 cumulative work가 증가한다.

[통합 추론] 2026년의 의미는 하나의 winner가 나온 것이 아니다. External learned consolidation, parametric fast→slow transfer, memory-harness evolution, within-query compaction, temporal multi-clock memory, modular LoRA knowledge memory, capacity와 rewrite의 negative result가 동시에 등장하면서 “memory algorithm”을 한 모델 내부 기능으로 보기 어려워졌다는 데 있다. 연구 대상은 scheduler, data builder, consolidator, destination router, verifier, publication protocol, serving cache, deletion graph를 합친 system으로 이동했다.

3. 생물학적 비유가 주는 것과 주지 않는 것

3.1 비교적 강하게 말할 수 있는 범위

[확립된 근거] 생물학적 수면과 기억의 관계에 관해 방어 가능한 최소 합의는 세 층이다. 첫째, wake 경험과 관련된 hippocampal ensemble·sequence activity가 NREM slow-wave/ripple 상태에서 재현된다. 둘째, [Rasch 등의 odor cue 재활성화](#), Girardeau 등의 sharp-wave ripple 방해, [Ngo 등의 phase-locked slow-oscillation stimulation](#)처럼 수면 중 특정 cue나 rhythm을 조작하면 이후 declarative 또는 spatial memory가 달라질 수 있다. 셋째, sleep은 단순 보관뿐 아니라 plasticity와 synaptic renormalization, 경험 통합과 연관된다.

이 근거는 “sleep이 기억에 무관하지 않다”와 “offline state의 dynamics가 이후 성능에 인과적으로 기여할 수 있다”를 지지한다. 그러나 AI 설계로 옮기는 순간 여러 추가 선택이 필요하다. 무엇을 replay할지, 어떤 representation을 사용할지, 어떤 objective로 바꿀지, 몇 번 돌릴지, 어느 state에 쓸지, 언제 멈출지는 neuroscience가 자동으로 답하지 않는다.

3.2 파일 이동 은유의 한계

[확립된 근거] 현대화된 CLS와 multiple-trace/transformation 관점은 hippocampus에서 cortex로 파일 하나를 복사하고 원본을 지우는 단순 그림보다 복잡하다. Replay는 quiet wake에도 일어난다. Schema-consistent 정보는 cortical network가 빠르게 배울 수 있다. 상세 episodic trace는 semantic gist가 cortex에 생긴 뒤에도 hippocampal dependency를 유지할 수 있다. 따라서 장기기억 전환은

copy → delete

보다

replicate → transform → re-index → dependency change

에 가깝다.

[통합 추론] 이 차이는 AI memory governance에 직접 연결된다. External event에서 summary가 만들어지고, summary에서 graph edge와 adapter가 파생됐다고 해서 원본 event의 역할이 끝난 것이 아니다. Correction, temporal query, rare one-off recall, legal deletion, poisoned promotion rollback에는 원본 또는 동등한 reversible substrate가 필요할 수 있다. 어떤 시점에 raw evidence를 지울 수 있는지는 “summary가 그럴듯한가?”가 아니라 future query family와 recovery guarantee를 기준으로 판단해야 한다.

3.3 NREM과 REM을 알고리즘 이름으로 쓸 때의 규율

[확립된 근거] PAD는 NREM-like reconstruction과 REM-like adversarial recombination을 배치했다. Singh-Norman-Schapiro 모델은 NREM에서 hippocampus-neocortex coupling, REM에서 neocortex-only replay를 구현했다. Google Sleep은 fast-to-slow Knowledge Seeding과 synthetic Dreaming을 분리했다. 세 방법은 모두 “보수적 보존”과 “탐색적 생성”을 구분하지만 objective, model, data, timing은 서로 다르다.

[비주장] 이 공통점으로부터 실제 뇌의 NREM이 distillation이고 REM이 counterfactual generation이라고 결론내릴 수 없다. REM dream report가 존재한다고 해서 generative augmentation이 생물학적으로 확정되는 것도 아니고, Crick-Mitchison 가설이 있다고 해서 REM pruning이 합의된 메커니즘이 되는 것도 아니다.

[제안 가설] 공학적으로 유용한 질문은 stage 이름이 아니라 objective conflict다. Conservative replay·verification은 exactness와 source fidelity를 높이려 하고, exploratory recombination은 coverage와 transfer를 넓히려 한다. 두 gradient나 verifier requirement가 충돌할 때만 stage separation의 이득이 나타날 수 있다.

이를 검증하려면 다음 순서를 compute-matched로 비교해야 한다.

```
replay/distillation only
novel generation only
replay → dream
dream → replay
interleaved joint objective
같은 추가 data와 compute를 쓴 random augmentation control
```

Exact retention, temporal correction, composition, robustness, hallucination, cycle-depth drift를 함께 봐야 한다. 어느 순서도 모든 regime에서 이기지 않거나, 단순 추가 data control이 같은 이득을 내면 NREM/REM-style stage claim은 좁혀져야 한다.

3.4 no-free-resurrection

[확립된 근거] 감사한 replay·sleep 시스템은 대개 완전히 지워진 기억을 무에서 되살리지 않는다. Robins에는 frozen old function이, DGR에는 generator와 old solver가, Krishnan/SRC에는 mean activation statistic과 residual weight가, Golden과 González에는 subthreshold synaptic trace가, PAD에는 latent buffer와 generator가, external memory에는 text/vector/graph object가 남아 있다.

전체 잔존 상태를

$$\Omega_t = (\theta_t, \phi_t, B_t, s_t, M_t, \text{metadata}_t)$$

라고 하자. 특정 과거 변수 Z 에 대해 $I(Z; \Omega_t) = 0$ 이고 새 randomness R 가 Ω_t 를 조건으로 Z 와 독립이면

$$I(Z; \Omega_t, R) = 0$$

이다. 독립적인 dream noise만으로 지워진 task-specific mapping을 정확히 복구할 수 없다. Correlated prior나 새 외부 관측이 정답을 추측할 수는 있지만, 그것은 retained trace의 복원과 다른 현상이다.

[통합 추론] “Sleep이 기억을 회복했다”는 결과를 읽을 때는 행동상 inaccessible했던 trace가 어디에 남아 있었는지를 찾아야 한다. 이 원칙은 memory-free claim과 deletion claim을 동시에 교정한다. Generator-only replay는 raw buffer가 없을 뿐 generator가 memory다. Source token을 KV에서 free한 뒤 answer가 유지됐다고 해서 semantic information이 지워진 것도 아니다.

[제안 가설] Recoverability margin이 0에 가까워지기 전에 intervention하면 같은 compute의 늦은 sleep보다 retention-plasticity frontier가 좋아질 것이다. Old-task canary margin, teacher agreement, source availability, retrieval confidence, representation conflict를 past-only proxy로 보정하고, predicted boundary 직전·직후·fixed cadence를 비교한다. Proxy가 held-out stream에서 보정되지 않거나 직전 intervention이 이기지 못하면 empirical scheduling hypothesis는 실패한다. 정보론적 no-free-resurrection 경계 자체와 proxy의 실패는 별개로 기록한다.

4. 두 축을 넘어서 기억 매질 지도

4.1 첫 번째 축: external인가 parametric인가

[통합 추론] 사용자 framing의 중심축은 정확하다.

외부기억

text / vector / graph / raw event / procedure / trajectory

모수적·잠재기억

KV / recurrent state / adapter / sparse slot / expert / fast weight / shared weight

External memory는 inspectability, provenance, correction, item-level delete, model portability에 유리하다. 대신 retrieval miss, false association, context token, index rebuild, stale/conflicting fact, archive growth 비용을 낸다. Parametric memory는 repeated use에서 prompt와 retrieval을 줄이고 representation-level generalization을 만들 수 있다. 대신 interference, addressability, routing, HBM residency, version explosion, selective unlearning이라는 비용을 낸다.

[확립된 근거] External memory가 stability-plasticity 문제를 제거하지 않는다는 반례가 존재한다. Retrieval context가 제한되면 old/new memories가 query attention을 두고 경쟁한다. ReasoningBank에서는 더 많은 memory를 주입할 때 성능이 낮아질 수 있었다. 반복 rewrite는 context collapse를 만들 수 있다. Graph에는 invalidated edge와 raw episode가 남고, active index를 줄여도 archive는 계속 증가할 수 있다.

[확립된 근거] Parametric destination도 retrieval-free가 아니다. Modular LoRA knowledge memory는 document별 adapter를 골라 load하거나 merge해야 한다. 같은 total trainable parameter에서 oracle-routed 여러 low-rank module이 single larger adapter보다 강할 수 있지만 learned router가 이득을 없애고 naive merge가 붕괴할 수 있다. 따라서 vector search를 module search로 바꾼 셈이며 catalog, compatibility, residency까지 포함해야 한다.

4.2 두 번째 축: explicit인가 implicit인가

External/parametric 이분법만으로는 MEMENTO, Cartridges, recurrent memory를 설명하기 어렵다. 다음 축을 더해야 한다.

```
explicit / inspectable ↔ implicit / latent
portable / re-prefillable ↔ execution-bound / stateful
canonical evidence ↔ derived materialized view
```

[확립된 근거] Microsoft [MEMENTO](#)는 동일한 memento text를 source-conditioned KV와 함께 사용할 때와 restart/re-prefill할 때 AIME24 pass@1이 66.1%와 50.8%로 달랐다고 보고했다. Source block에만 넣은 opaque passcode가 downstream KV에서 chance 이상으로 복원되기도 했다. Visible summary가 완전한 state가 아니라는 직접 증거다.

[통합 추론] 따라서 latent artifact의 manifest에는 단순 파일 경로 외에 model/tokenizer, attention mask, position convention, KV layout, source snapshot, serialization class가 들어가야 한다.

```
PORTABLE_TEXT
SERIALIZED_LATENT
LIVE_KV_LINEAGE
REBUILD_ONLY
```

Text가 같아도 restart equivalence가 없으면 다른 artifact다. Model upgrade 뒤 재사용 가능성, source deletion 뒤 residual information, peak KV와 memory-time AUC를 따로 측정해야 한다.

4.3 canonical과 derived를 구분하는 이유

[통합 추론] Raw event는 retention policy가 허용하는 동안 system of record다. Summary, embedding, graph, latent, adapter, shared weight는 모두 source에서 만든 materialized view다. Derived view는 빠르고 유용할 수 있지만 source의 모든 기능을 보존한다고 가정할 수 없다.

Canonical이 필요한 이유는 네 가지다.

1. **정확한 correction과 temporal reconstruction.** 현재 값뿐 아니라 무엇이 언제 supersede됐는지 알아야 한다.
2. **rebuild와 representation migration.** Encoder나 base model이 바뀌면 old latent 또는 embedding을 재생성해야 한다.
3. **poison rollback과 audit.** 어떤 source가 어떤 artifact에 영향을 줬는지 추적해야 한다.
4. **rare-query recoverability.** Summary가 버린 one-off detail을 늦게 묻는 query가 올 수 있다.

반대로 raw archive를 영원히 보존하는 것도 정답이 아니다. Privacy, storage, attack surface, deletion obligation이 증가한다. 필요한 것은 archive depth와 reversibility reserve를 명시하고, 줄일 때 어떤 guarantee를 포기하는지 선언하는 것이다.

4.4 promotion/demotion ladder

[제안 가설] 하나의 안전한 기본 policy는 다음 사다리다.

```
raw event / exact transcript
  ↓ admission, deduplication, scope assignment
episodic text·vector·temporal graph
  ↓ merge, abstraction, contradiction handling
semantic/procedural external memory
  ↓ context distillation or compilation
latent KV·cartridge·user adapter·fast weight
  ↓ repeated validated reuse + replay/distillation
slow cohort/shared parametric memory
```

새 사건은 처음부터 weight로 쓰지 않는다. Novelty, importance, confidence, expected reuse, volatility, privacy scope를 추정한 뒤 evidence가 약하면 exact external에 둔다. 여러 session에서 다른 표현의 query에 계속 도움이 되고 correction이 드물며 serving 비용을 줄인다는 근거가 생기면 KV나 user adapter로 compile한다. Shared promotion은 cross-user reuse와 authorization이 별도로 검증될 때만 허용한다.

Demotion은 실패가 아니다. Reuse가 떨어지거나 correction/delete risk가 오르면 adapter를 retire하고 external evidence로 돌아갈 수 있다. Shared weight에서 item-level demotion이 불가능하면 affected module을 quarantine하거나 canonical source로 clean rebuild해야 한다.

[제안 가설] Positive switching cost와 monotone single-crossing value curve가 존재하면 promotion threshold와 demotion threshold를 분리한 hysteresis가 thrashing을 줄일 것이다. 그 조건이 성립하지 않으면 universal two-threshold law를 주장하지 않고 empirical policy surface로 남겨야 한다.

4.5 제품과 논문을 같은 표준으로 읽기

[통합 추론] MemGPT는 online virtual context와 tiered store의 출발점이다. Letta paper는 pre-query token compilation이다. Letta sleeper는 product-level async core-memory editing이다. Mem0 paper/OSS는 mostly wake 경계의 candidate fact extraction과 ADD/UPDATE/DELETE/NOOP이며, managed Platform V3만 비동기 queue를 쓰되 감사된 core path는 ADD-only다. Graphiti OSS는 temporal entity/fact ingestion과 invalidation, optional community abstraction을 제공하지만 scheduler를 소유하지 않는 PARTIAL이다. Zep Cloud의 asynchronous construction은 PASS이고, Managed Zep Observations는 공개 OSS core와 다른 proprietary operator다.

이들을 모두 “장기기억을 sleep에서 학습한다”로 묶으면 세 오류가 생긴다. 첫째, weight가 변했는지 prompt와 store만 변했는지 사라진다. 둘째, current query path인지 deferred path인지 사라진다. 셋째, paper version과 현재 product behavior가 섞인다.

[통합 추론] 모든 시스템을 같은 lifecycle schema로 재기술해야 한다.

```
input event
timing boundary
operator
destination and scope
learned or prompted signal
source lineage
capacity and expiry
delete semantics
publication and rollback
evaluation query relationship
```

이 schema를 통과하면 vendor name보다 비교 가능한 object가 남는다.

4.6 Google, Meta, Microsoft의 서로 다른 위치

[통합 추론] Google/DeepMind 계보는 CLS·EWC의 이론·continual basis, Titans–Miras/Atlas의 wake neural memory, Nested Learning의 multi-frequency view, Google Sleep의 explicit parametric consolidation으로 이어진다. ReasoningBank는 success/failure trajectory를 external bank에 append하는 parallel wake branch다. Memory Caching, TNT, NSTM은 cache, update frequency, training efficiency, offload를 다루는 systems branch다. 모두 하나의 직선 dependency는 아니며 최초 공개 chronology와 현재 paper dependency를 구분해야 한다.

[통합 추론] Meta 계보는 GEM·MAS의 stability, Product-Key Memory와 Expire-Span의 sparse lookup·lifetime, Memory Layers와 SMF의 sparse parametric memory로 이어진다. 감사된 공개 근거 안에서 strict async sleep pipeline은 확인되지 않는다. 따라서 Meta의 기여를 “sleep infrastructure”보다 “유한 sparse capacity와 저간섭 write”로 읽는 편이 정확하다.

[통합 추론] Microsoft 계보는 LongMem의 external latent bank, Generative Adapter의 context→LoRA, PlugMem·MemMA·HIMA의 external transform, MEMENTO의 within-query state compaction, M★·RHO의 memory harness search, MemCompiler의 delivery compiler, LEGOMem의 procedural bank 등 병렬 branch다. HIMA는 scheduled external sleep에 직접적이고, RHO는 completed trajectories 이후 persistent harness를 바꾸는 strict experimental boundary를 가진다. 그러나 Letta와 같은 token-space scaling law나 Google Sleep과 같은 실험적 fast→slow parametric consolidation pipeline을 Microsoft 전체의 단일 계보로 부르면 안 된다. Google Sleep 자체도 production/deployed lifecycle이 확인된 것은 아니다.

5. Sleep에서 무엇을 학습하는가: data, objective, policy

5.1 완전한 sleep pipeline

[통합 추론] Sleep-time training은 optimizer 이름이 아니라 다음 factorization이다.

```
wake events
→ admission and immutable snapshot
→ replay / dream / abstraction corpus
→ transformation or training operator
→ external / latent / parametric candidate
→ retention + acquisition + governance validation
→ publish / quarantine / rollback
```

문헌에는 각 조각의 precedent가 있지만, 미래 query utility, bounded lifetime capacity, plasticity, correction, deletion, rollback을 한 번에 해결한 방법은 없다. 그러므로 “최고의 sleep loss”를 찾기보다 data, operator, destination, cadence를 분리해 실험해야 한다.

5.2 canonical replay와 작은 anchor

[통합 추론] Raw replay는 source fidelity가 가장 높고 correction·delete·audit가 쉽지만 storage, privacy, I/O가 비싸다. Tiny Episodic Memories는 평가한 continual-learning benchmark에서 class당 한 exemplar만으로도 큰 benefit을 보였다. SRC는 intrinsic replay에 0.75% real rehearsal를 더했을 때 크게

개선됐다. Facts in Weights는 forgotten fact가 context로 다시 주어진다면 recover될 수 있음을 보이며 external canonical copy를 권고했다. Auto-Dreamer도 active memory region을 교체하면서 provenance-linked source trajectory를 유지한다.

[통합 추론] Generator-only를 기본으로 삼기보다 작은 stratified canonical anchor를 함께 두는 것이 보수적 baseline이다. Anchor는 무작위로만 뽑지 않는다. Old, rare, negative, correction, high-delete-risk, boundary case를 포함해야 한다. 다만 future utility를 직접 사용해 뽑은 oracle coreset은 deployable baseline이 아니며 upper bound로 따로 둔다.

5.3 bounded coreset의 선택 문제

Replay buffer policy는 서로 다른 bias를 만든다.

- Uniform reservoir는 stream의 평균 분포를 보존하지만 rare high-value item을 놓칠 수 있다.
- Class-balanced sampling은 majority domination을 줄이지만 class label이 필요하고 open-ended label space에서 계속 자란다.
- Recent+historical mixture는 plasticity와 stability를 섞지만 old distribution이 stale할 수 있다.
- Gradient-constraint buffer는 관측된 old loss를 직접 보호하지만 constraint solve와 exemplars가 비용이다.
- Surprise·novelty gate는 duplicate write를 줄이지만 threshold drift와 rare-known false rejection을 만든다.
- Value+diversity selection은 중복을 줄일 수 있지만 value estimator가 틀리면 irrecoverable selection bias가 된다.

[확립된 근거] Surprise-Gated Memory의 recent-five-task replay가 no-replay보다 나뉘었다는 결과는 “fixed window면 안전하다”를 반박한다. Full-history mixture가 보호한 것은 old-support distribution이었다. Bytes를 줄이는 것과 support geometry를 지키는 것은 다르다.

[제안 가설] Fixed-byte coreset이 성공하려면 age, class/task diversity, rare correction, protected-old canary를 함께 보존해야 한다. uniform, recent, class-balanced, surprise-only, utility-diversity, stratified canonical을 같은 byte와 lifetime replay work에서 비교한다. Buffer 크기만 같고 full-history replay 횟수가 다르면 compute-matched 비교가 아니다.

5.4 latent replay와 representation expiry

[확립된 근거] SIESTA는 MobileNetV3 latent를 OPQ로 압축했고, FearNet은 Gaussian class statistics를 사용했으며, PAD는 latent를 occlude·mix했다. Cartridges는 corpus를 recurrent/KV state로 compile했다. Latent replay는 raw I/O와 active bytes를 줄일 수 있지만 encoder dependency를 만든다.

Encoder가 바뀌면 old latent가 unreadable하거나 의미가 달라질 수 있다. Encoder를 영원히 freeze하면 새 domain에 대한 plasticity가 제한될 수 있다. Canonical source가 있으면 re-embed할 수 있지만 index refresh cost가 생긴다. Source를 지웠으면 old encoder를 보존하거나 translation·federation을 운영해야 한다.

[통합 추론] Text/vector memory에도 역방향 mismatch가 있다. Sleep이 payload text를 rewrite했는데 old embedding을 유지하면 retrieval key와 payload가 다른 generation에 속한다. Payload digest, key digest, encoder digest, authorization, provenance를 하나의 manifest에서 atomic하게 publish해야 한다. Fixed encoder의 local rewrite는 changed item만 refresh하면 되지만 incompatible global encoder change는 single-current-encoder contract에서 whole-store re-embed를 요구할 수 있다.

5.5 generated replay와 dream data

[확립된 근거] Generated replay는 세 유형으로 나뉜다.

1. DGR·DGDMN의 pseudo-input과 old solver target.
2. FearNet·PLOS의 distributional latent/statistical replay.
3. SEAL·Google Sleep의 query, self-edit, curriculum generation.

어느 경우도 memory-free가 아니다. Generator parameters, teacher checkpoint, statistic, prompt, seed, rejected sample, generation compute 중 일부가 past information과 비용을 운반한다.

Recursive self-teaching에는 다음 경로가 생긴다.

```
teacher error
→ generated target bias
→ student drift
→ next-cycle teacher error
```

Facts in Weights의 frozen original teacher와 recursively updated teacher 차이는 이 위험을 직접 경고한다. Sleep data manifest에는 teacher가 이미 self-updated됐는지, cycle depth, input distribution, hard/soft target mixture가 들어가야 한다.

[통합 추론] 좋은 dream corpus의 최소 조건은 다음과 같다.

1. Old, recent, rare, failed, corrected event의 coverage.
2. Paraphrase뿐 아니라 counterfactual과 cross-episode composition.
3. Source span, simulator, tool outcome에 의한 grounding.
4. Canonical, teacher, student-on-policy, adversarial, random sample의 anti-collapse mixture.
5. Generator와 충분히 독립적인 verifier.
6. Privacy, poison, contradiction, impossible counterfactual rejection.
7. Dream likelihood가 아닌 held-out future utility.

[제안 가설] Generator replay에는 nonzero canonical-anchor floor가 있다. 평균 accuracy는 anchor fraction을 줄여도 부드럽게 변할 수 있지만 rare-event survival과 recoverability는 어느 지점에서 급격히 무너질 수 있다. Generator-only, 작은 canonical anchor, larger anchor를 cycle depth와 함께 sweep해 이 phase boundary를 찾는다.

5.6 supervised replay, distillation, regularisation

[통합 추론] Supervised replay는 label을 직접 사용하는 강한 baseline이다. SIESTA, WSCL, ordinary rehearsal가 여기에 속한다. Sleep-inspired method는 같은 canonical bytes, old/new ratio, optimizer updates, augmentation, checkpoint, task information, lifetime compute에서 이 baseline을 이겨야 한다.

Distillation은 source record가 아니라 teacher behavior를 옮긴다. Progress & Compress는 active column을 knowledge base로, DGR은 old solver를 new solver로, Cartridges는 full-context teacher를 KV state로, Google Sleep은 fast expert를 new slow expert로 옮긴다. Teacher가 memory이므로 identity, version, input distribution, KL direction, temperature, coefficient, reset order를 공개해야 한다.

Regularisation과 gradient constraint는 old behavior를 보호하지만 saturation을 지연할 뿐 제거하지 않는다. EWC/SI는 importance와 anchor state를 저장하고 protected direction을 늘린다. GEM은 examples와 constraint solve를 저장한다. Old retention이 정상이어도 new acquisition이 떨어질 수 있으므로 retention-only evaluation은 부족하다.

[제안 가설] Parametric sleep의 강한 baseline은 canonical replay + distillation + isolated adapter다. Generator-only 또는 shared-weight update가 이를 이기려면 같은 total state에서 better future utility를 보이고, protected slice와 acquisition, deletion, rollback을 함께 통과해야 한다.

5.7 sparse write, isolation, growth, recycle

[확립된 근거] Adapter, expert, sparse row, hard mask, progressive column은 interference를 줄인다. SMF는 new batch에 특이적인 memory row만 update한다. Google Sleep은 masked slow expert를 활성화한다. PackNet은 pruning 뒤 free weights를 후속 task에 배정한다. Continual Backpropagation은 low-utility unit을 작은 비율로 재초기화해 long-run plasticity를 완화했다.

이 방법들은 capacity allocation이지 infinite capacity가 아니다. 다음 수량을 구분해야 한다.

- active
- protected
- free
- verified recyclable
- quarantined
- archived
- preallocated but inactive

Google Sleep의 finite expert pool이 차면 merge, prune, grow, retire 중 하나를 해야 한다. Adapter rank가 고정돼도 adapter count, catalog, optimizer, merge workspace, HBM copies, checkpoints는 자란다.

[제안 가설] Plasticity-first alarm은 old retention보다 먼저 binding constraint를 알려 줄 수 있다. Fresh control과 같은 data·step budget을 주고 current model의 acquisition gain을 비교한다. Acquisition 감소, free/recyclable fraction, representation stable rank, sleep backlog를 결합한 signal이 recycle/rebuild benefit을 예측하는지 검증한다. Recycle은 canonical replay 없이 쓰면 old memory를 파괴할 수 있으므로 restricted low-utility recycle + canonical replay, random recycle, no recycle를 분리한다.

5.8 external-memory transformation도 학습이다

External consolidation은 gradient가 없어도 memory operator다. Mem0의 ADD/UPDATE/DELETE/NOOP, Graphiti의 entity resolution과 temporal invalidation, PlugMem의 proposition·prescription induction, MemMA의 probe→repair, HIMA의 tier promotion·decay, Evolve의 section compile, DreamDaemon의 topic rewrite가 그 예다.

[통합 추론] External operator의 학습 문제는 다음처럼 바뀐다.

어떤 event를 admit할 것인가?
어떤 records를 coalesce할 수 있는가?
무슨 exact evidence를 남겨야 하는가?
어떤 contradiction이 old fact를 닫거나 version할 것인가?
어떤 future-query utility가 abstraction을 정당화하는가?
delete가 모든 derived node에 어떻게 도달하는가?

Active index를 bounded하게 만들어도 raw archive, invalidated graph, provenance, candidates가 unbounded일 수 있다. Prompt에 넣는 memory가 적다고 total state가 작은 것도 아니다.

[확립된 근거] MemMA는 session이 끝난 뒤 five synthetic probes로 provisional store를 검사하고 실패를 fact repair로 바꾸는 패턴을 보여 준다. 그러나 paper-aligned probe generator가 공개되지 않았고 한 LoCoMo conversation에 한정된다. Probe는 평가 query가 아니라 learner-visible derived training artifact다. Generator code, prompt, source snapshot, accepted/rejected probe, seed를 lineage에 포함하고 final evaluation과 물리적으로 분리해야 한다.

5.9 memory architecture와 harness를 학습하는 단계

[확립된 근거] MemEvolve는 Encode/Store/Retrieve/Manage code를 Pareto search했고, MemSkill은 PPO selector와 최대 30개의 natural-language skills를 학습했으며, ALMA는 database/update/retrieve design을 탐색했다. M★는 schema, write/read logic, instruction을 함께 진화시켰다. RHO는 completed trajectories를 난도·다양성으로 고르고 반복 solve·diagnosis를 거쳐 instruction, skill, executable tool 전체를 proposal하고 pairwise self-preference로 winner를 선택했다.

이 계보는 memory content뿐 아니라 **기억을 해석하고 쓰는 프로그램**도 learned state라는 점을 보여 준다. 그러나 search iteration이나 coreset 크기가 고정돼도 selected harness가 만드는 store, code version, source trajectory, diagnosis, backup, run log는 계속 자랄 수 있다. RHO의 one-round search는 selection 뒤에도 103 agent invocations를 요구하며 solver, diagnostician, proposer, ranker가 같은 model family다. Positive self-score는 independent safety attestation이 아니다.

[제안 가설] Workload heterogeneity가 커질 때 universal harness와 full task-specific rewrite 사이에 frozen governance kernel + specialised write/read module이라는 중간 optimum이 존재할 수 있다. 같은 trajectories와 artifact budget에서 세 arm을 비교하고, independent promotion judge, sandbox, poison canary, migration, exact rollback을 모두 charge한다. Full rewrite 또는 universal harness가 모든 scale에서 우월하면 중간 계층 가설은 반증된다.

5.10 sleep policy를 위한 RL의 네 층

[통합 추론] RL은 하나의 “sleep RL”이 아니라 네 control level에 존재한다.

1. **Data-generation policy.** SEAL은 self-edit와 synthetic data를 만들고 post-update reward로 policy를 학습한다. Google Sleep Dreaming은 candidate question을 생성·filter하고 downstream outcome으로 ReSTEM을 수행한다.
2. **Replay-selection/orchestration policy.** Spens의 recurrent PPO는 bounded offline window에서 replay, world-model update, generated learning, no-op을 고른다. MSSR은 loss-derived memory strength로 replay interval과 ratio를 조절하지만 continual training loop 안의 wake comparator다.
3. **Consolidation-operator policy.** Auto-Dreamer는 retain/replace에 가까운 external transform을 end-to-end future reward로 학습한다. TrustMem은 write/revise/prune/no-op candidate를 coverage, preservation, faithfulness 관점에서 고른다.
4. **Retrieval/delivery policy.** MEMORA의 REFINE/EXPAND/STOP, MemCompiler의 text+soft-token compiler는 stored memory를 current action에 전달하는 wake policy다.

[통합 추론] 실제 sleep reward는 낮고 부분 관측이며 미래 workload에 의해 confounded된다. 단일 next-task success만 최적화하면 rare long-horizon memory와 delete obligation을 희생할 수 있다. 후보 reward는

$$R = w_u U_{\text{future}} - w_f F_{\text{protected}} - w_h H_{\text{false}} - w_c C_{\text{lifecycle}} - w_g G_{\text{governance}}$$

처럼 쓸 수 있지만, 모든 항을 사전 정의한 공통 unit으로 정규화할 때만 scalar 의미가 있다. 그렇지 않으면 multi-objective constraint다. Logged propensity나 simulator가 없으면 off-policy estimate를 confirmatory evidence로 쓰지 않는다.

5.11 cadence와 trigger

[확립된 근거] 문헌에는 short-term slot full, every 10 sessions, every 120K samples, task boundary, task midpoint, completed task, completed session, fixed multi-frequency transfer, default six hours, manual/03:00, 600-second scan 등 많은 trigger가 존재한다. 어느 것도 universal optimum을 확립하지

않았다.

[통합 추론] Trigger는 한 scalar clock보다 pressure vector여야 할 가능성이 높다.

novelty arrival과 duplicate rate
retrieval competition과 stale/conflict
retention margin과 acquisition decline
free slot/expert fraction
sleep queue와 retry load
privacy/delete deadline
idle window와 accelerator availability

TiMem식 session/day/week/month clock은 active context를 줄일 수 있지만 construction call을 늘린다. Recurrence trigger는 one-off를 억제하지만 희귀 중요 사건을 놓칠 수 있다. Surprise는 novelty signal이지만 threshold drift가 rare-known memory를 잘못 거부할 수 있다. Trigger와 admitted data mixture를 함께 평가해야 한다.

[제안 가설] Fixed wall clock보다 $\text{memory pressure} \times \text{calibrated novelty} \times \text{expected reuse} \times \text{current service load}$ 를 쓰는 scheduler가 나올 수 있다. 단, learned trigger overhead와 instability가 fixed cadence보다 크면 반증된다. Privacy deletion과 poison quarantine처럼 deadline이 있는 job은 learned utility보다 hard policy가 우선한다.

6. 용량, 망각, plasticity: 무엇이 먼저 차는가

6.1 memory capacity는 적어도 다섯 개다

[통합 추론] 용량 문헌이 말하는 capacity는 하나의 양이 아니다.

1. **Storage capacity**는 몇 byte, vector, edge, slot, expert, parameter state를 물리적으로 보유하는지 묻는다.
2. **Retention/representational capacity**는 update가 계속될 때 old trace가 recall threshold 위에 얼마나 남는지 묻는다.
3. **Addressability/application capacity**는 정보가 남아 있어도 query, router, retriever, reader가 latency와 context budget 안에서 실제로 찾고 쓰는지 묻는다.
4. **Plasticity/acquisition capacity**는 old retention이 정상인 상태에서도 동일 data·step budget으로 새 것을 fresh control만큼 배울 수 있는지 묻는다.

5. **Governance capacity**는 provenance, consent, temporal correction, selective delete, rollback, backup expiry를 처리할 수 있는지 묻는다.

여기에 sleep service throughput과 HBM hot residency가 운영 capacity로 붙는다. “Weight에 남아 있다”, “active store가 줄었다”, “old accuracy가 유지됐다”는 서로 다른 관찰이며 서로를 증명하지 않는다.

6.2 total state와 peak state

[통합 추론] Incremental answer-bearing state의 최소 회계는

$$B_{\text{retained}} = B_r + B_e + B_l + B_p + B_a + B_v$$

이다. B_r 은 raw/canonical evidence, B_e 는 text/vector/graph와 index, B_l 은 latent/KV, B_p 는 adapter/expert·trainable delta, B_a 는 optimizer·importance-router-generator, B_v 는 version·lineage·recovery다. Inactive expert, masked slot, catalog, teacher checkpoint도 실제 retained state면 포함한다.

Retained bytes만으로 build 순간의 hard cap을 알 수 없다.

$$B_{\text{peak, state}}(t) = B_{\text{retained}}(t) + B_{\text{candidate}}(t) + B_{\text{pinned}}(t) \\ + B_{\text{migration}}(t) + B_{\text{workspace}}(t) + B_{\text{replica}}(t).$$

Candidate staging, reader가 붙잡은 predecessor, dual index, merge workspace, temporary replica는 모두 disjoint excess byte로 센다. 각 항에는 hard cap과 GC owner가 필요하다. Cumulative transformation work는 byte에 더하지 않고 별도 ledger로 둔다.

Frozen base는 incremental information bound의 public side information일 수 있지만 infrastructure에서는 공짜가 아니다.

$$B_{\text{peak, physical}} = \max_t \left[B_{\text{base, resident}}(t) + B_{\text{peak, state}}(t) + B_{\text{execution}}(t) \right].$$

세 항은 각각의 독립적인 최대값을 더하는 것이 아니라 동일한 maximizing timestamp t 에서 측정한다.

Separate wake/sleep fleet는 contention을 줄일 수 있지만 base model 복제와 network movement를 늘린다. Scientific capacity claim에는 incremental state, cluster claim에는 physical peak를 모두 공개해야 한다.

6.3 유한 상태의 정보 경계

[확립된 근거/필요조건] Frozen public protocol과 base를 θ_0 , n 개 event 뒤 모든 incremental mutable state를 M_n 이라 하자. M_n 은 external record, weight delta, latent, generator, index, optimizer, provenance, recovery copy를 모두 포함한다. Finite precision 아래

$$H(M_n | \theta_0) \leq b_{\text{mutable}}$$

이고, controlled answer Y_i 가 선언한 source model 아래 조건부 독립이며 distortion target이 D_i 라면 joint queryability에는

$$\sum_{i=1}^n R_{Y_i | Q_i, \Theta_0} (D_i) \leq I(Y_{1:n}; M_n | Q_{1:n}, \Theta_0) \leq b_{\text{mutable}}$$

가 필요하다. 자연어 fact 수에 대한 universal theorem이 아니라 opaque independent payload처럼 entropy와 query를 통제할 실험의 necessary bound다.

[통합 추론] Positive-entropy novel event가 일정 rate로 계속 들어오는데 mutable state가 고정되고 distortion을 일정하게 유지한다면 언젠가 모순이 생긴다. 시스템은 결국 하나 이상을 해야 한다.

retained state를 늘리거나 다른 tier로 옮기고 비용을 센다.
진짜 redundancy와 prior를 이용해 필요한 rate를 낮춘다.
admission 또는 retained horizon을 제한한다.
summary·expiry·forgetting으로 distortion을 높인다.

Bounded hot set와 growing cold archive는 serving residency만 제한할 뿐 lifetime storage를 제한하지 않는다. Reconstruction service가 archive를 다시 만들 수 있다면 그 service와 seed도 state다.

6.4 bounded synapse가 주는 이론적 경계

[확립된 근거] Fusi–Abbott는 bounded efficacy와 ongoing random plasticity에서 trace가 overwrite되며, state precision을 늘리는 이득이 potentiation/depression balance에 민감함을 보였다. Lahiri–Ganguli는 finite-state Markov synapse의 ideal-observer bound에서 internal state complexity도 capacity에 포함해야 함을 보였다. Benna–Fusi는 bidirectional fast/slow variables와 여러 time scale이 initial strength와 lifetime의 frontier를 바꿀 수 있음을 보였다.

[비주장] 이 논문들의 $N / \log N$, $\sqrt{NM}$, power-law trace를 transformer factual capacity에 직접 대입할 수 없다. Synapse 하나를 parameter, vector row, expert와 일대일 대응할 근거도 없다. 이론의 정당한 역할은 변수와 countermodel을 제안하는 것이다. 즉 multi-timescale state가 공짜가 아니며, biased update와 finite precision이 중요하고, one-way copy보다 bidirectional coupling을 실험할 가치가 있다는 정도다.

6.5 catastrophic forgetting과 loss of plasticity

[확립된 근거] Catastrophic forgetting은 old example을 다시 보지 않을 때 old performance가 떨어지는 문제다. Loss of plasticity는 long-run training 뒤 새 objective를 배울 능력 자체가 낮아지는 문제다. [Dohare 등의 연구](#)는 continual setting에서 dormant unit 증가와 stable-rank 감소, new-task learning deterioration을 보고했고 low-utility unit의 지속적 작은 재초기화가 이를 완화했다.

Old-task recall만 측정하면 plasticity가 먼저 무너진 상태를 성공으로 오판할 수 있다. Fixed-budget acquisition gain을

$$g_{\text{acq}} = \frac{U(\theta \text{ after}) - U(\theta \text{ before})}{U(\theta_{\text{fresh}} \text{ after}) - U(\theta_{\text{fresh}} \text{ before})}$$

처럼 fresh control에 정규화할 수 있다. Fresh denominator가 learnability threshold보다 작으면 그 cell은 0으로 채우지 말고 invalid로 처리한다.

[제안 가설] 적어도 하나의 high-interference regime에서 acquisition boundary가 aggregate retention boundary보다 먼저 온다. 이 순서가 어떤 frozen regime에서도 나타나지 않으면 plasticity-first hypothesis는 반증된다. Restricted recycle이 acquisition을 회복하지 못하거나 protected old slice를 해치면 intervention hypothesis는 별도로 실패한다.

6.6 replay work와 recursive distillation의 누적

[확립된 회계] Task t 뒤 모든 과거 task $i \leq t$ 의 m_i sample을 replay하면

$$R_t = \sum_{i=1}^t m_i, \quad W_T = \sum_{t=1}^T R_t = \sum_{i=1}^T m_i (T - i + 1).$$

$m_i = m > 0$ 이면 lifetime replay count는 $\Theta(T^2)$ 다. Surprise gate가 constant fraction만 admit하면 coefficient는 줄지만 exponent는 그대로다. Latest- w window는 $O(wT)$ 로 줄지만 recent-only counterexample처럼 retention을 더 악화시킬 수 있다.

매 k write마다 현재 n 개 fact 전체를 redistill하는 방식도 대략 $O(n^2/k)$ lifetime work를 만든다. 매 cycle fixed-size narrative를 재작성하면 published state는 $O(1)$ 이어도 lifetime work는 최소 선형이다. Bounded output, bounded active view, bounded total state, bounded selection work는 네 개의 다른 주장이다.

6.7 retrieval competition과 coverage

[확립된 근거] External memory가 커질수록 oracle-store survival, retriever recall, reader use 사이의 간극이 커질 수 있다. 더 많은 item을 context에 넣는 것이 utility를 단조롭게 높이지 않는다. Exact repeated acquisition query의 cache hit는 held-out paraphrase, composition, correction, distribution shift의 semantic reuse와 다르다.

Deployment query mix를 relationship class r 로 나누면

$$h_{\text{deploy}}(M) = \sum_r \rho_r h_r(M)$$

이다. Exact repeat만 반복한 실험은 h_{exact} 를 측정할 뿐 deployment mix를 식별하지 못한다. Evolve의 cold/warm/post가 같은 250 questions를 같은 순서로 반복한 결과는 exact-key amortization의 근거이며 unseen semantic transfer의 근거가 아니다.

[조건부 이론] Relevant state support가 compact metric space이고 effective covering dimension d 를 가지며 fixed codec·bytes per prototype 조건에서 approximately optimal cover를 만든다면 coverage radius는

$$r_M = \Theta \left(M^{-1/d} \right)$$

후보다. 그러나 total bit budget을 고정하면 item 수 M 을 늘릴수록 payload precision이 줄어

$$r_{\text{eff}}(M, B) \lesssim cM^{-1/d} + q(b_{\text{payload}}(M))$$

같은 geometry-quantization trade-off가 생긴다. Open-ended drift, anisotropic relevance, merge distortion, rising retrieval error가 있으면 깨진다. 이는 external-memory workload의 조건부 candidate이지 language memory의 universal law가 아니다.

6.8 parametric bit capacity와 offloading

[확립된 근거] Allen-Zhu와 Li의 약 2 knowledge bits/parameter, Morris 등의 약 3.6 memorized bits/parameter는 저장 object와 readout, training protocol이 다르다. 하나의 universal constant에 대한 두 추정치가 아니다. Natural-language fact의 entropy도 일반적으로 알려져 있지 않다.

[제안 측정] Opaque independent payload를 사용하면 conditional parametric efficiency를

$$\hat{c}_\theta = \frac{\hat{I}(Y; \theta | Q, \theta_0)}{\Delta P}$$

bits/added parameter로 정의할 수 있다. 각 seed에서 θ_0 부터 다시 train하고, decoder orientation을 CALIBRATION에서 freeze하며, TEST 뒤 decoder를 바꾸지 않는다. Payload seed, dataset, replay log, auxiliary decoder가 answer를 운반하면 complete mutable-state denominator에 들어간다. Base model이 seed로 payload를 재생성하는 fixture는 parametric memory가 아니라 hidden external state다.

[확립된 근거] LMLM과 LaCy는 factual target을 external call로 돌리는 pretraining allocator를 보여 줬다. LaCy의 중요한 null은 factual offloading이 tested NLU를 유의하게 개선하지 않았다는 점이다. “Fact bit를 빼면 같은 양의 reasoning capacity가 자동으로 생긴다”는 conservation law는 없다.

[제안 가설] Fixed parameter, gradient-token, external-call budget에서 selected factual target을 외부화했을 때 fresh non-factual acquisition이나 plasticity가 실제로 늘어야 “usable capacity release”를 주장할 수 있다. Factual leakage만 줄고 acquisition이 그대로면 simple freed-reasoning-capacity mechanism은 그 regime에서 반증된다.

6.9 네 포화 모드와 대응

[통합 추론] 관측 가능한 포화는 다음처럼 구분된다.

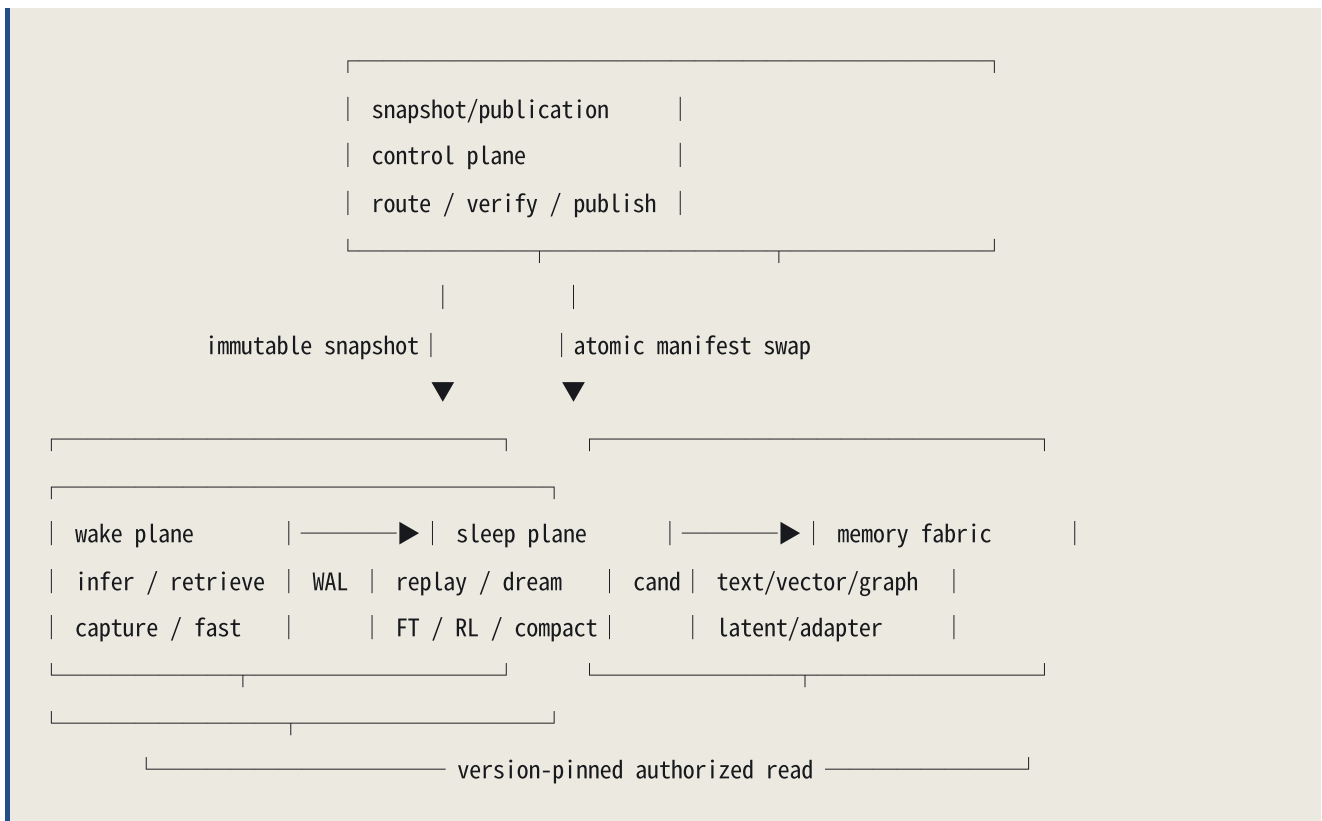
- **물리적 포화:** archive, graph, adapter, expert, checkpoint, optimizer가 증가한다. Dedup, tiering, coreset, quantization, offload, verified recycle, deliberate forgetting이 필요하다.
- **간접 포화:** source confusion, old loss, gradient overlap, slot collision, repeated-consolidation damage가 증가한다. Interleaved replay, sparse isolation, projected gradient, external-first promotion, clean rebuild가 후보 operator다.
- **Plasticity 포화:** fixed-budget acquisition, representation rank, free capacity가 낮아진다. Low-utility recycle, shrink, optimizer reset, adapter rollover, fresh-base reconstruction을 canonical replay와 결합한다.
- **Governance 포화:** correction과 deletion이 derived artifacts에 전파되지 않고 rollback versions와 delete debt가 쌓인다. Dependency DAG, tombstone, authorization epoch, rebuild/unlearn, backup expiry가 필요하다.

[통합 추론] 한 operator는 압력을 없애기보다 이동시킨다. Replay는 weight interference를 data/service pressure로, sparse write는 interference를 free-slot pressure로, externalization은 weight interference를 retrieval/governance pressure로, separate cluster는 wake contention을 data movement와 duplicate residency로 바꾼다. 좋은 system은 이 이동을 감추지 않고 같은 ledger에서 보여 준다.

7. Wake-sleep-장기기억의 통합 시스템

7.1 세 execution · state plane과 snapshot/publication control plane

[제안 아키텍처]



세 execution·state plane은 wake, sleep, memory다. Wake는 latency-sensitive execution과 fast state, sleep은 throughput-oriented transformation, memory plane은 tiered persistent state를 담당한다. 이를 가로지르는 snapshot/publication control plane이 immutable event range, policy, consent·deletion epoch, candidate validation, atomic publication, rollback을 담당한다. 이 logical responsibility 구분이 곧 physical three-cluster를 뜻하지 않는다. 같은 fleet에서 priority isolation할 수도 있고 물리적으로 분리할 수도 있다.

7.2 wake plane

Wake plane은 inference, retrieval, prompt assembly, optional fast/session state, append-only event capture를 담당한다. User response SLO가 최우선이다. Event log에는 opaque source handle, tenant/privacy scope, event/ingest time, payload reference, tool outcome, model/prompt version, correction·consent·retention epoch가 들어간다.

Wake가 수행할 수 있는 online learning은 명시한다.

```

raw event append
bounded session summary
fast-state statistic
user-local adapter step
contradicted graph edge invalidation
  
```

이들은 sleep이 아니라 wake/test-time baseline이다. Sleep이 나중에 consolidate하거나 undo할 수 있도록 parent state와 operation digest를 남긴다.

7.3 snapshot과 serving head

[제안 아키텍처] Tenant의 publication authority는 하나의 linearizable record여야 한다.

$\text{ServingHead} = (\text{manifest generation}, \text{manifest digest}, \text{authorization epoch}, \text{deletion epoch})$.

네 field를 따로 확인하면 validation과 publish 사이 authorization race가 생긴다. Sleep job은 exact parent head, event range $[a, b]$, policy digest, budget, input digests를 받는다. 이후 event는 다음 snapshot에 남는다. Deletion이나 authorization change는 content reclaim이 끝나기 전이라도 head epoch를 올려 immediate deny를 선형화한다.

Sleep은 live tree를 in-place mutate하지 않는다.

- temporary objects
 - checksums와 readiness
 - candidate manifest
 - validation attestations
 - exact-parent compare-and-swap
 - serving head 교체

Crash나 stale worker가 있어도 all-new 또는 all-old만 보이고, lost event 없이 idempotent replay하며, predecessor rollback이 가능해야 한다.

7.4 sleep plane의 job class

Sleep worker는 하나의 generic trainer가 아니다.

- extract / redact / embed
- diagnostic probe와 repair
- dedup / merge / abstract / rewrite
- temporal rollup과 local graph maintenance
- view compilation과 index refresh
- replay shard와 dream shard generation
- distillation / SFT / adapter packaging
- policy RL 또는 memory-program search
- rebuild / verify / delete propagation

각 job은 GPU-s, CPU-s, HBM-byte-s, RAM-byte-s, I/O bytes, network bytes의 native resource vector를 선언한다. FLOPs 하나로 verifier, storage, network, residency를 숨기지 않는다.

Scheduler action은 NO_OP, RETAIN_EXACT, EXPIRE, PROBE_REPAIR, DEDUP, MERGE, ABSTRACT, REWRITE, GRAPH_UPDATE, LATENT_COMPILE, ADAPTER_COMPILE, PARAMETRIC_PROMOTE, DEMOTE, RECYCLE, REBUILD, QUARANTINE처럼 operator와 destination을 분리한다. Learned policy가 제안하더라도 privacy, authorization, deletion, hard cap, promotion gate는 학습 policy 밖의 kernel이다.

7.5 candidate validation

Candidate와 incumbent는 같은 probes에서 비교한다.

current와 future utility
protected old retention
fresh-normalized acquisition
exact / semantic / temporal / relational / procedural recall
false memory와 calibration
rare / failure / correction slice
poison, privacy, authorization
delete dependency completeness
latency, state, compute, movement
rollback recovery

Aggregate gain은 protected slice violation을 상쇄하지 못한다. Retention과 acquisition은 별도 gate다. Diagnostic probe generator는 observable snapshot만 보고 final evaluation query를 보지 못해야 한다.

7.6 heterogeneous state의 atomic publication

External memory는 vector store, relational metadata, topic file, graph root, index root, source staging range를 동시에 바꿀 수 있다. 각각의 write 성공을 atomic이라고 부르면 안 된다. Candidate manifest가 모든 sub-root와 referential integrity를 묶고, readiness barrier 뒤 단 하나의 serving pointer를 CAS해야 한다.

Changed payload는 retrieval key를 recompute하거나 equivalence를 증명해야 한다. Encoder digest mismatch는 sampled cosine이 우연히 작아도 실패다. Latent/adapter는 model, tokenizer, shape, precision, source, scope, compatibility가 일치해야 한다. Publication receipt가 나온 뒤에만 source staging range를 acknowledge한다.

[제안 가설] Snapshot+canary+atomic publication+rollback은 in-place update보다 median cost를 조금 늘리더라도 worst-case utility loss와 mixed-generation error를 크게 줄일 것이다. Fault injection에서 tail-risk gain이 lifecycle cost를 상쇄하지 못하면 transactional-compaction advantage는 반증된다.

7.7 long-term memory fabric

Memory fabric은 scope와 medium의 두 axis를 갖는다.

```
scope: shared fixed / shared slow / cohort / user / session / query
medium: raw / text / vector / graph / latent / adapter / weight / auxiliary
```

Hot tier는 wake SLO를 위한 active retrieval set과 resident adapters다. Warm tier는 user/tenant text, vector, graph, frequently loaded modules다. Cold tier는 canonical event, older versions, rebuild source, rollback checkpoint다. Tier를 옮겼다고 총 bytes가 사라지는 것은 아니다.

Adapter registry는 source lineage, tenant scope, base/tokenizer compatibility, rank/shape, training data digest, route key, merge set, load/swap cost, rollback expiry를 기록한다. Merged adapter는 모든 input lineage를 유지하며 selective delete 가능성을 별도 검증하지 않으면 그렇게 광고할 수 없다.

7.8 deletion과 correction

[통합 추론] Delete는 단일 Boolean이 아니다.

1. 즉시 authorization deny
2. in-flight descendant quarantine
3. online payload와 identifier mapping 제거
4. text/vector/graph/latent view invalidate 또는 rebuild
5. adapter retire/rebuild/unlearn
6. backup/key expiry
7. surviving KV/recurrent state의 opaque residual probe
8. bytes, lineage, behavior audit

Access denial과 physical erasure, behavioral forgetting을 따로 보고한다. Shared weight에서 exact item removal을 가정하지 않는다. High-delete-risk personal data의 shared promotion을 막고 user adapter에 격리하며, 필요 시 module quarantine 또는 clean rebuild를 수행한다.

Correction은 old assertion의 validity interval, new assertion, source confidence, affected descendants, serving precedence, rebuild receipt를 남긴다. Graph node나 summary를 덮어쓰기만 하면 temporal query와 audit evidence를 잃는다.

7.9 data movement와 physical fleet

[통합 추론] 목표는 infra 사이 이동을 무조건 없애는 것이 아니라 utility당 불필요한 이동을 줄이는 것이다. Wake는 full history 대신 append-only delta와 sufficient statistics를 보낸다. Sleep은 full checkpoint 대신 graph patch, summary delta, sparse slot diff, LoRA를 반환한다. Raw evidence는 cold object store, index와 summary는 warm service, active adapter와 KV는 HBM에 둔다.

네 physical option을 비교한다.

1. **Shared priority fleet:** wake가 sleep을 preempt한다. Idle capacity와 weight reuse가 좋지만 P99 interference와 checkpoint thrash가 위험하다.
2. **Time-partitioned fleet:** 예측 가능한 저부하 시간에 role을 바꾼다. Diurnal valley가 뚜렷할 때 유리하지만 global timezone과 unfinished jobs가 문제다.
3. **Separate fleets:** latency serving과 batch training을 분리한다. Hard isolation과 sustained sleep load에 유리하지만 base duplication과 state movement를 낸다.
4. **Shard-local hybrid:** extract-compact는 data 근처, heavy distillation은 pooled GPU, 작은 delta만 귀환한다. Privacy와 large raw source에 유리하지만 orchestration이 복잡하다.

[제안 가설] Separate cluster가 항상 최적이라는 권리는 없다. Sleep load, wake interference, model size, source shard size, output delta, batching, network, privacy locality를 sweep해 crossover를 찾아야 한다. Data locality도 raw source 이동 절감이 model residency·batching 손실보다 큰 regime에서만 이득이다.

7.10 queue와 운영 capacity

Compute, flow, occupancy를 단위별로 분리한다.

$$\sum_j \lambda_j E[w_j^{\text{gpu}}] < m_{\text{gpu}}, \quad \sum_j \lambda_j E[d_j^{\text{io}}] < C_{\text{io}}, \quad \sum_j \lambda_j E[a_j^{\text{hbm}}] < B_{\text{hbm, cap}}.$$

첫째는 GPU count, 둘째는 byte/s, 셋째는 byte stock이다. RAM bytes를 storage bandwidth와 비교하지 않는다. Mean 조건 외에 instantaneous HBM/RAM cap, p99 job age, privacy deadline, tenant starvation을 봐야 한다.

Session/day/week/month clock은 서로 다른 job class다. Requested, coalesced, provider/model call, completed publication을 따로 센다. Clock 수 증가를 aggregated call 하나로 숨기지 않는다.

[제안 가설] High-novelty burst에서는 physical store보다 queue freshness가 먼저 실패할 수 있다. Backlog age가 cap hit보다 먼저 SLO를 위반하면 memory capacity의 첫 knee는 storage가 아니라 sleep service다. Admission, cheaper operator, local region, defer, retain-exact로 대응하되 silent unbounded backlog를 허용하지 않는다.

8. 제안 scaling laws와 claim-down 규율

8.1 법칙이라고 부르기 전의 네 표지

현재 문헌에는 external, latent, parametric, governance, infra를 함께 덮는 deployed STC scaling law가 없다. 아래 관계는 성격을 구분한다.

- **IDENTITY**: 정의한 accounting boundary에서 항상 성립하는 회계식.
- **NECESSARY**: 선언한 queue·information assumptions 아래 필요조건.
- **DERIVED-CANDIDATE**: 제한된 stationary model에서 유도한 optimum 또는 threshold.
- **EMPIRICAL-CANDIDATE/HYPOTHESIS**: held-out scale과 workload에서 검증할 함수형.

[통합 추론] 이 구분을 지우면 analytic special case가 universal law로 과장된다. Scaling program의 목표는 하나의 예쁜 exponent보다 어느 조건에서 어떤 bottleneck이 바뀌는지를 예측하는 것이다.

8.2 reuse break-even

Compile cost C_{compile} , maintenance C_{maint} , use당 saving $\Delta C_{\text{use}} > 0$ 라면 restricted break-even은

$$n^* = \frac{C_{\text{compile}} + C_{\text{maint}}}{\Delta C_{\text{use}}}.$$

Reuse가 Poisson rate λ_i , invalidation hazard μ_i , discount ω 라면 infinite-horizon valid reuse는

$$R_i(\infty) = \frac{\lambda_i}{\mu_i + \omega}.$$

[제안 가설] High reuse·low volatility의 stable procedure는 adapter/KV compilation이 이득이고, low reuse·high correction/delete risk의 fact는 external이 이득일 것이다. 단일 threshold는 workload마다 curve가 여러 번 교차하거나 query type이 지배하면 거부한다. Letta의 multi-query saving을 one-off query에 일반화하지 않는다.

8.3 cadence law

Run cost C_s , delay penalty $a\tau^p$ 의 restricted stationary model은

$$\bar{C}(\tau) = \frac{C_s}{\tau} + a\tau^p, \quad \tau^* = \left(\frac{C_s}{ap} \right)^{1/(p+1)}$$

를 준다. Linear staleness special case는 event rate r_e , per-event harm h , fixed cost F 에 대해

$$\tau^* = \sqrt{\frac{2F}{hr_e}}$$

다.

[비주장] Bursty arrival, batching economy, deadline, repeated-consolidation damage, teacher drift, capacity phase transition에서는 깨진다. 따라서 analytic policy, segmented trace-driven policy, learned trigger를 비교하고 universal “몇 시간마다”를 주장하지 않는다.

8.4 byte-flow equilibrium

Store j 의 physical byte identity는

$$\frac{dB_j}{dt} = r_{\text{admit},j} \bar{b}_{\text{in},j} - r_{\text{expire},j} \bar{b}_{\text{delete},j} - r_{\text{compact},j} \bar{b}_{\text{reclaim},j}.$$

Active store에서 archive로 옮긴 byte는 physical reclaim이 아니다. One-cycle merge ratio가 양수여도 net addition이 더 크면 total store는 자란다. Bounded lifetime claim에는 nondegenerate novel stream에서 장기 평균 admission minus physical reclamation이 0 이하라는 flow evidence와 utility·provenance 유지가 필요하다.

8.5 capacity knee

Retention threshold $\tau_{\text{ret}} = 0.80$ 을 stake로 freeze한 후보 logistic은

$$\text{logit } p_{\text{ret}}(N_a) = \text{logit } \tau_{\text{ret}} - \kappa \ln(N_a/K_{\text{eff}})$$

이다. K_{eff} 는 $p_{\text{ret}} = 0.80$ 이 되는 association load다. Exact와 temporal 두 component 모두 low-load simultaneous lower bound > 0.90 , high-load upper bound < 0.70 인 bracket을 CALIBRATION에서 정한다.

[제안 규율] Eight points와 finite coefficient만으로 knee를 주장하지 않는다. Monotonic decrease, single crossing, residual envelope를 통과하고 shape-constrained sensitivity의 crossing이 0.10 log-capacity 안에서 동의해야 한다. Multiple crossing이나 model disagreement가 크면 KNEE_UNIDENTIFIED다.

Separate knee를 추정한다.

- physical storage
- retention/representation
- addressability/application
- plasticity/acquisition
- governance/recoverability
- sleep service
- hot-state residency

첫 binding knee가 현재 regime을 정의하며 intervention 뒤 이동할 수 있다.

8.6 four-axis capacity surface

Reference resource tuple r_0 에서 workload family f , medium m 의 anchor를

$$K_0 (f, m) = K_{\text{eff}} (f, r_0, m)$$

로 둔다. 후보 surface는

$$\frac{K_{\text{eff}}}{K_0} \propto \left(\frac{B_{\text{store}}}{B_0} \right)^\alpha \left(\frac{T_{\text{live}}}{T_0} \right)^\beta \left(\frac{F_{\text{sleep}}}{F_0} \right)^\gamma \left(\frac{b_{\text{repr}}}{b_0} \right)^\nu$$

다. Controlled axes는 active external/store cap, live token cap, sleep-compute cap, serialized latent+adapter bits다. Realized HBM, raw archive, physical peak, actual FLOPs는 constraint/outcome이며 hidden fifth input으로 바꾸지 않는다.

Power, min-bottleneck, generalized mean, segmented knee, interaction, monotone nonparametric model을 경쟁시킨다. Named held-out workload는 fit을 다시 조정하지 않는다.

8.7 modular-parametric routing knee

Fixed total adapter parameter에서

$$\sum_{k=1}^K \Delta P (r_k) = \Delta P_{\text{single}}$$

를 맞춰도 catalog, router, cache, optimizer, recovery byte는 별도다. Oracle utility U_{oracle} 와 deployed router·merge·residency utility U_{deploy} 의 aggregate tax는

$$L_{\text{module}} = U_{\text{oracle}} - U_{\text{deploy}}$$

다.

[제안 가설] Isolation gain이 router error, merge interference, selection, HBM miss보다 클 때만 multi-module이 single adapter를 이긴다. Rank, module count, query sparsity, composition demand, catalog residency를 sweep한다. Learned router가 held-out composition에서 oracle advantage를 회복하지 못하면 modular region은 그 regime에서 반증된다. Routing·merge·residency를 post-hoc으로 임의 분해하지 않고, 별도 factorial을 G2 전에 등록했을 때만 attribution한다.

8.8 clock, catalog, harness, refresh의 scaling

[제안 가설] 네 가지 새 systems crossover가 중요하다.

- **Clock-count crossover:** temporal level을 늘리면 active byte당 utility가 처음에는 좋아지지만 construction calls, queue age, repeated abstraction distortion가 커지면 역전된다.

- **Catalog-selection crossover:** 모든 skill description을 prompt에 넣는 flat selector는 K 에 따라 token이 정확히 합산된다. Hierarchical router는 build·refresh·miss recovery 비용을 내며 어느 catalog size와 reuse 이상에서만 이득이다.
- **Harness-specialisation crossover:** universal harness, shared kernel+task module, tenant-specific, per-task full rewrite 사이에서 heterogeneity와 drift가 search·migration·governance cost를 상쇄하는 지점을 찾는다.
- **Representation-refresh amplification:** fixed encoder local rewrite는 affected item에 비례하지만 single-current-encoder/no-federation contract에서 incompatible encoder를 자주 바꾸고 store가 선형 증가하면 whole-store refresh가 quadratic lifetime work가 될 수 있다. Federation과 translation은 resident old encoder, fan-out, calibration, delete 비용을 charge해야 한다.

8.9 lifecycle frontier

Primary outcome은 QA accuracy 하나가 아니다.

future utility와 retention/acquisition
wake latency
sleep compute
retained/peak bytes
peak residency와 memory-time AUC
bytes moved
false memory
delete/rollback completeness
resource failure

Stakeholder exchange rate가 정당화되지 않으면 scalar “memory score”를 만들지 않는다. 같은 information, model, hardware, governance, missing-run policy에서 모든 frozen axis가 no worse이고 적어도 하나가 better일 때만 Pareto dominance를 주장한다.

8.10 claim-down rules

[제안 규율]

- A100 configuration과 workload-family holdout만 통과하면 **A100-regime scaling law**다.
- Prospectively powered second hardware holdout까지 통과해야 **cross-hardware/system law**다.
- Named family에서만 맞으면 **regime-specific relation**이다.
- Fit은 되지만 holdout prediction이 실패하면 **empirical curve**다.
- Alternative가 구분되지 않거나 precision이 부족하면 **null/narrowed**다.
- Logistic adequacy가 없으면 finite knee를 내지 않고 **KNEE_UNIDENTIFIED**다.

- Active-store reduction은 total compression이 아니다.
- Behavioral forgetting은 physical deletion이 아니다.
- Exact-repeat saving은 semantic amortization이 아니다.
- Simulation은 absolute hardware latency나 energy evidence가 아니다.
- Product document는 product behavior evidence이지 comparative effectiveness evidence가 아니다.

Holdout failure 뒤 exponent 의미나 claim scope를 바꾸지 않는다. Sub-threshold statistical effect는 material support가 아니다. Equivalence interval이 margin과 겹치면 equivalence가 아니라 insufficient precision이다.

9. 검증 프로그램: 이름이 아니라 operator를 비교한다

9.1 benchmark가 내려야 하는 결정

[제안 실험] 같은 accumulated wake events와 future queries를 받은 시스템이 다음 중 무엇을 해야 하는지를 결정해야 한다.

exact external evidence를 보존한다.
external evidence를 merge/abstract한다.
reusable latent/KV로 compile한다.
user-local adapter로 compile한다.
deliberately forget 또는 no-op한다.
past-only router가 위 action을 동적으로 고른다.

Benchmark의 중심 질문은 past-only hybrid가 tuned single-medium과 static mixture보다 held-out lifetime utility/lifecycle-cost frontier를 개선하는가다. Clairvoyant oracle router는 heterogeneity value의 diagnostic upper bound이며 deployable method가 아니다.

9.2 lifetime cycle

한 user의 기본 horizon은 128 cycle이다.

- wake capture
 - pre-sleep probe
 - immutable snapshot
 - sleep/no-sleep action
 - candidate validation and promotion
 - post-sleep probe
 - delayed next-wake probe

Horizon sensitivity는 8, 32, 128, 512 cycle, retrieval lag는 1, 8, 32, 128 cycle을 사용한다. Session, event, query를 independent sample로 오인하지 않고 user/stream을 top-level unit로 둔다.

9.3 observable과 oracle의 물리적 분리

Method가 볼 수 있는 observable에는 opaque ID, event payload, 현재 time·confidence·privacy·provenance, 이미 알려진 correction/delete/poison flag, past reuse와 outcome, 현재 resource·queue state가 들어간다.

Oracle에는 true validity interval, future reuse·query distribution, future contradiction·deletion, poison truth, exact answers, clairvoyant destination이 들어간다. File, schema, process permission으로 분리한다. Future leakage mutation test는 oracle field를 바꾸어도 deployable configuration과 current action이 변하지 않는지 확인한다.

TRAIN은 learned router와 compiler를 fit한다. CALIBRATION은 baseline hyperparameter, cadence, threshold, comparator map, early stop을 freeze한다. TEST는 G2 뒤 한 번만 열고 baseline이나 claim wording을 선택하지 않는다.

9.4 event와 query family

Controlled stream은 stable exact facts, volatile preference, procedure/skill, temporal relation, failure/negative feedback, privacy/deletion, poison, one-off distractor를 독립적으로 변화시킨다. Opaque payload subfamily는 independent random bit를 사용해 conditional entropy를 알 수 있게 하고, 같은 semantic payload의 surface length를 바꾸어 information과 text byte를 분리한다.

Query는 exact, semantic, temporal, relational, procedural, negative/failure, calibration, deletion을 포함한다. Reusable item은 다음 관계 label을 가진다.

ACQUISITION_KEY
EXACT_REPEAT
HELDOUT_PARAPHRASE
HELDOUT_COMPOSITION
CORRECTION_PROBE
UNRELATED_CONTROL

Acquisition query를 반복해 얻은 saving은 semantic transfer endpoint에 들어가지 않는다.
Exact·temporal·executable query는 deterministic scoring을 우선하고 model judge는 secondary validity track으로 둔다. Writer와 grader가 같은 model/config이면 correlated evidence로 보고한다.

9.5 confirmatory method와 resource envelope

Deployable primary arms은 여섯 개다.

1. Versioned raw external.
2. Frozen merge operator를 쓰는 compressed external.
3. Architecture-compatible latent/KV compiler.
4. User-local LoRA compiler.
5. CALIBRATION에서 tuned한 static action mixture.
6. Past-observable history만 쓰는 hybrid router.

Graph, second latent implementation, shared-weight promotion, evolved harness, recurrent fast weight는 characterization 또는 prospective extension이다. 결과를 본 뒤 primary set에 넣지 않는다.

모든 method는 같은 upper envelope를 받는다.

live token cap
active·durable byte cap
total incremental peak-state cap
physical HBM/fleet peak cap
sleep-FLOP와 peak accelerator cap
same information cutoff
same privacy/delete contract

Unused budget을 보고한다. Search, router training, teacher/judge call, embedding, optimizer, implicit KV, index rebuild, archive, lineage, rollback, staging, migration, workspace를 모두 센다. Bytes moved, latency, energy는 역지로 equalize하지 않고 outcome으로 측정한다.

9.6 핵심 factorial과 논리적 run

Routing/medium track은 6 deployable methods $\times$ 3 reuse regimes $R \in \{1, 4, 16\} \times$ 3 active-capacity ratios $\phi \in \{1, 1/4, 1/16\} \times$ 3 paired stream-seed families로 162 logical runs다. 같은 27 operating cells의 oracle가 27을 더한다.

Schedule/operator track은 inline-immediate와 deferred-cycle-boundary, identity와 semantic consolidation, 세 reuse, 세 pressure, 세 seed family를 교차해 108 runs다. 합계 297 logical confirmatory lifetime cells다. Cell 안 user count는 blinded paired power simulation으로 결정한다. Physical execution을 재사용할 수 있어도 contrast와 multiplicity는 고정한다.

9.7 primary endpoint와 claim spine

각 cycle의 pre/post/delayed 세 probe block을 동등 가중해 lifetime utility AUC를 만든다. 한 absolute utility point는 normalized scale의 0.01이다. Primary comparison은 hybrid 대 G2-frozen benefit comparator와 efficiency comparator다. TEST에서 가장 좋은 baseline을 다시 고르는 max-envelope는 금지한다.

네 paper-level claim은 서로 다른 질문을 맡는다.

- **PAPER-C1:** clairvoyant heterogeneity value가 적어도 한 reuse $\times$ pressure cell에서 0.01을 넘는가.
- **PAPER-C2:** past-only router가 frozen comparator보다 utility +0.02 또는 utility noninferiority -0.01 와 lifecycle-cost reduction 10%를 함께 달성하는가. Resource-failure noninferiority가 별도 veto다.
- **PAPER-C3:** reuse와 pressure가 external $\rightarrow$ latent 또는 latent $\rightarrow$ user-parametric의 matched-cap utility ordering을 reversal시키는가. Cost는 feasibility와 diagnostics이며 utility crossover를 바꿔 쓰지 않는다.
- **PAPER-C4:** semantic consolidation operator가 material utility gain을 내고, identity에서는 deferred execution이 utility-equivalent하면서 cost를 줄이며, schedule $\times$ operator interaction이 equivalence margin 안에 드는가.

[제안 규율] Positive branch의 일부만 맞으면 “거의 support”라 하지 않는다. C4에서 operator gain은 있지만 interaction equivalence가 없으면 schedule-contingent/narrowed다. Every eligible operator UCB가 0.02 아래일 때만 no-operator-effect falsification이다. Precision 부족과 effect 부재를 구분한다.

9.8 retention과 acquisition의 분리

매 cycle old protected query retention과 fixed-budget new-task acquisition을 동시에 측정한다.

Parametric high-interference extension은 no recycle, restricted low-utility recycle+canonical replay, random recycle+same replay, fresh-base diagnostic을 비교한다.

Secondary neural diagnostics는 dormant unit, activation/gradient stable rank, weight magnitude, free/recyclable fraction, optimizer age다. 이들은 behavioral acquisition endpoint를 대신하지 않는다.

9.9 statistical contract

Confirmatory target은 16 workload families $\times$ 3 paired-seed families의 48 fixed equal-weight blocks다. 각 block 안 user/stream을 resample하고 48 blocks 자체를 IID superpopulation처럼 resample하지 않는다. 모든 powered row는 적용 가능한 workload $\times$ seed block마다 최소 4 independent users를 갖는다.

Primary uncertainty에서 104개 continuous scientific margin은 exactly 20,000 deterministic blocked paired bootstrap replicate와 48-block-stratified paired-stream-cluster sandwich studentization을 사용한다. 27개 resource-failure companion margin은 boundary-positive paired-discordance score를 사용한다. 두 종류의 statistic은 PAPER-C-DEPLOYABILITY-GLOBAL-131 하나의 joint max-statistic으로 함께 보정한다. Claim-specific critical value나 uncentered statistic으로 support를 쉽게 만들지 않는다.

99,999-draw wild-cluster sensitivity와 leave-one-seed-family-out direction이 primary decision과 불일치하면 support하지 않는다. Blinded pilot은 covariance와 sample size를 정하지만 scientific margin을 바꾸지 않는다. Pilot covariance builder 자체도 hierarchy, bounded support, shrinkage, PSD projection, stress coverage를 pre-pilot에 freeze한다.

9.10 missingness와 resource failure

모든 method를 함께 망가뜨린 shared protocol corruption만 block invalidation 후 MNAR bound 대상이다. Method timeout, OOM, deadline miss, cap violation, compiler exception, invalid artifact, absent probe는 observed RESOURCE_FAILURE다. Randomized stream을 유지하고 consumed work를 charge하며 affected probe는 0점으로 처리한다. Complete-case analysis는 sensitivity-only다.

Intentional ABSTAIN/FORGET가 protocol-valid COMPLETE 상태일 때만 abstention credit을 받을 수 있다. 실패한 method가 “기억하지 않기로 했다”고 사후 해석할 수 없다.

9.11 failure injection과 artifact graph

다음 fault를 주입한다.

candidate write 전후 worker kill
truncated object와 index corruption
vector/metadata의 엇갈린 commit
old key + new payload 또는 new key + old payload
staging clear before manifest CAS
stale-out-of-order publication
deletion during pinned read
model/encoder upgrade
queue overload
rollback predecessor authorization loss
concurrent foreground query

모든 run은 source/version → question/hypothesis → config/data/code/environment → raw checksum → derived result → claim → figure/table → manuscript sentence로 추적한다. 결과 figure보다 artifact graph가 먼저 완전해야 한다.

10. Open questions와 검증 가능한 가설

10.1 현재 답할 수 있는 것

[확립된 근거]

- Sleep은 operational phase로 정의할 수 있고 strict delayed transform의 precedent가 존재한다.
- Replay, distillation, generated data, sparse isolation, external merge, learned offline action은 모두 2025년 이전 또는 동시기에 선행한다.
- External과 parametric destination 모두 실재하며 어느 하나도 capacity와 interference를 제거하지 않는다.
- Generated replay는 memory-free가 아니며 teacher/generator state와 error debt를 가진다.
- Retention과 plasticity는 다른 failure mode다.
- NREM/REM role assignment은 유용한 computational factorization이지만 확립된 일대일 biological mapping이 아니다.

10.2 matched experiment가 필요한 질문

[제안 가설]

아래 항목 가운데 H-STC-004, H-STC-006, H-STC-007의 현재 G2 허용 상태는 derivation, property test, preregistration contract뿐이다. 다음 문장은 미래 확장 설계와 반증 규칙이며 empirical result가 아니다. 효과, interval, support/falsification 같은 empirical status는 별도의 prospectively powered G2 amendment가 manifest, power, multiplicity, receipt, validation을 결속하고 실행된 뒤에만 부여할 수 있다.

1. **Representation shift (H-STC-001).** Future-query distortion가 exact, semantic, procedural로 바뀌면 optimal medium ordering도 바뀐다. Query distribution을 바뀌도 ordering reversal이 없으면 반증된다.
2. **Reuse-volatility regions (H-STC-002).** Reuse와 invalidation hazard가 conditional destination crossover를 만든다. Monotone single-crossing 조건이 충족됐는데 held-out에서 predicted crossing이 없으면 반증한다. 조건 자체가 실패하면 claim을 좁힌다.
3. **Hysteresis (H-STC-003).** Positive switching cost 아래 separated promotion/demotion threshold가 thrashing을 줄인다. Sufficient condition이 성립한데도 threshold separation 또는 policy gain이 없으면 반증한다.
4. **Binding capacity (H-STC-004, 미래 확장 계약).** CALIBRATION에서 binding으로 진단된 일곱 capacity cell 중 적어도 하나는 +25% resource intervention으로 utility를 0.01 이상 높이고, nonbinding intervention은 ± 0.005 equivalence 안에 든다. 최소 네 eligible cells가 없으면 antecedent inconclusive다.
5. **Reversibility reserve (H-STC-005).** Archive depth는 rollback value와 storage·attack·delete cost 때문에 적어도 한 eligible regime에서 interior optimum을 가진다. 충분한 range의 모든 regime가 monotone이면 반증한다.
6. **Recoverability scheduling (H-STC-006, 미래 확장 계약).** Past-only proxy가 trace 파괴 전 intervention timing을 개선한다. Calibration 실패 또는 matched before-boundary action의 무이득이면 실패다.
7. **Plasticity-first (H-STC-007, 미래 확장 계약).** 적어도 한 high-interference regime에서 acquisition이 aggregate retention보다 먼저 실패한다. 아무 regime에서도 순서가 나타나지 않으면 반증한다.

10.3 장기 deployment measurement가 필요한 질문

[통합 추론] Universal cluster split, cadence hour, archive depth, user/cohort/shared scope는 production arrival, privacy, hardware, model residency에 의존한다. Benchmark가 조건부 crossover를 줄 수는 있어도 전 세계 서비스의 optimum을 확정할 수 없다. 특히 128-cycle success는 indefinite reliability가 아니다.

10.4 탐색적 확장 가설—독창성·우선권 미확립

다음은 confirmatory outcome을 본 뒤 승격할 수 없는 exploratory agenda다.

- **Canonical-anchor boundary:** generator-only가 rare/recovery slice에서 급락하는 최소 anchor fraction.

- **Teacher-corruption curve:** recursive teacher가 frozen 또는 refreshed teacher보다 cycle-depth error를 빠르게 쌓는가.
- **Stage-count crossover:** fast/medium/slow tier의 추가가 metadata, copy error, read amplification을 이기는 범위.
- **Governance shadow price:** correction/delete probability가 external versus parametric routing boundary를 얼마나 이동시키는가.
- **Queue-before-storage:** novelty burst에서 backlog freshness가 physical cap보다 먼저 binding되는가.
- **Explicit/implicit completeness gap:** compression pressure와 hop depth가 restart utility와 latent privacy gap을 키우는가.
- **Repeated-key/semantic gap:** exact cache saving과 held-out paraphrase·composition·correction saving의 차이.
- **Catalog-selection tax:** injected skill 수는 작아도 selector prompt·route·compatibility cost가 전체를 지배하는 지점.
- **Coverage-dimension law:** fixed-codec $M^{-1/d}$ 와 fixed-bit geometry-quantization trade-off가 held-out에서 예측되는가.
- **Representation-refresh amplification:** encoder evolution과 affected-set size가 index lifetime work를 어떻게 바꾸는가.
- **Bounded-state/lifetime-work separation:** fixed published narrative가 rewrite work를 숨기는 crossover.
- **Usable-capacity release:** fact offloading이 fresh non-factual acquisition을 실제로 높이는가.
- **Parametric-routing knee:** module isolation benefit이 router·merge·residency tax와 만나는 지점.
- **Clock-count queue crossover:** temporal hierarchy의 utility가 service와 abstraction debt를 넘는 지점.
- **Internalization-completeness gap:** direct recall parity가 update, reference, composition, relevance, boundary-awareness parity를 과장하는가.
- **Recovery-substrate floor:** raw/reversible state를 없앤 뒤 rare correction·late query·rollback에 남는 nonzero error floor.

11. 이 연구를 반증하는 결과

NM급 연구는 “무엇이 나오면 멋진가?”보다 “무엇이 나오면 주장을 접는가?”를 먼저 적어야 한다.

11.1 sleep-specific benefit의 반증

Delayed snapshot arm이 compute·data·destination을 맞춘 immediate wake update보다 utility, wake latency, lifecycle cost, safety, amortization 어느 축에서도 낮지 않으면 해당 operator의 sleep-specific claim을 철회한다. “더 많은 정보를 나중에 봤다”는 arm은 timing effect가 아니라 information accumulation effect다.

11.2 hybrid superiority의 반증

Past-only router가 four single media와 static mixture의 frozen comparator를 material margin으로 이기지 못하거나 resource-failure veto를 통과하지 못하면 deployable hybrid superiority는 실패한다. Oracle만 강하면 heterogeneity는 존재하지만 learnable routing evidence는 없는 것이다.

11.3 NREM/REM factorization의 반증

Replay→dream, dream→replay, interleaved design이 같은 extra data/compute control을 이기지 못하거나 stage ordering이 model·workload마다 불안정하면 biological-stage naming을 제거하고 objective mixture의 empirical result로 낮춘다.

11.4 capacity law의 반증

Knee model이 monotonic single crossing과 residual test를 통과하지 못하거나 shape-constrained crossing과 동의하지 않으면 K_{eff} 를 보고하지 않는다. Four-axis surface가 held-out scale 또는 family를 예측하지 못하면 exponent를 law라 부르지 않는다. Cross-hardware holdout이 없으면 A100 regime 밖으로 일반화하지 않는다.

11.5 canonical-anchor hypothesis의 반증

Generator-only가 same total state와 compute에서 rare, correction, deletion, cycle-depth holdout까지 canonical-anchor hybrid와 동등하고 recoverability gap이 없으면 nonzero anchor floor는 그 regime에서 반증된다. “Raw를 안 썼다”가 아니라 generator와 seed를 포함한 complete state가 정말 맞아야 한다.

11.6 plasticity-first hypothesis의 반증

현재 H-STC-007은 derivation/property/preregistration contract이며 empirical status가 없다. 별도의 prospectively powered G2 amendment와 실행 evidence가 갖춰진 뒤에만 다음 판정을 적용한다. 모든 preregistered high-interference regime에서 acquisition과 retention이 동시에 또는 retention 먼저 실패하면 plasticity-first ordering은 반증된다. Diagnostic rank가 변해도 behavioral acquisition이 유지되면 rank를 controller로 승격하지 않는다.

11.7 transactional advantage의 반증

Snapshot, canary, atomic publication, rollback이 fault-induced tail loss를 material하게 줄이지 못하고 median/lifecycle overhead만 키우면 해당 workload에서 in-place 또는 simpler transaction이 우월하다. Transactional architecture는 안전 원칙이라도 empirical cost-benefit claim은 실패할 수 있다.

11.8 semantic amortization의 반증 또는 지지

Exact repeat와 held-out paraphrase·composition·correction에서 quality-floor-conditional teacher-call saving이 동등하면 repeated-key gap hypothesis는 반증된다. 반대로 exact에서만 saving이 크면 “general memory compilation”이 아니라 cache-like result로 claim을 낮춘다.

11.9 bounded lifetime의 반증

Active view가 줄어도 archive, versions, graph, adapter catalog, implicit KV, backups, rewrite work, queue age 중 하나가 제한 없이 증가하면 bounded lifetime claim은 실패한다. Finite horizon에서 cap을 넘지 않았다는 사실만으로 asymptotic equilibrium을 주장하지 않는다.

11.10 deletion의 반증

Tombstone 뒤 retrieval이 실패해도 physical replica, derived graph, adapter, backup, surviving KV에서 source information이나 behavioral influence가 남으면 complete deletion은 실패다. 해당 stage별로 denied, online removed, derived invalidated, backup pending, behavioral residual unresolved를 따로 보고한다.

12. 단계별 연구 agenda와 산출물

12.1 1단계: canonical evidence와 vocabulary 고정

산출물.

- 최초 공개일, venue, version, affiliation, code/data를 분리한 chronology.
- 각 연구의 wake input, timing, operator, destination, scope, capacity, delete, rollback을 기록한 operator matrix.
- Paper, product, repository behavior를 분리한 source ledger.
- strict sleep, wake learning, pre-query compilation, adjacent method 판정.

이 단계의 성공은 논문 수가 많아지는 것이 아니라 같은 이름의 다른 mechanism을 더 이상 섞지 않는 것이다.

12.2 2단계: deterministic local reproducibility

작은 open model과 controlled stream으로 event generator, oracle separation, exact scoring, capacity ledger, snapshot/CAS, fault injection을 재현한다. 외부 raw, compressed, latent, user-LoRA, static mixture, simple router의 최소 구현을 동일 interface에 올린다.

산출물.

- Deterministic lifetime stream과 leakage test.
- Sleep-data manifest와 source lineage.
- Total retained/peak/physical state instrumentation.
- Delete and rollback receipts.
- Exact-repeat versus semantic query split.

12.3 3단계: matched confirmatory experiment

Blinded pilot, frozen comparator, global multiplicity, 297 logical cells, 48 fixed blocks를 실행한다. Positive claim뿐 아니라 null branch를 manuscript 구조에 미리 둔다.

산출물.

- PAPER-C1-C4 결과와 adjusted intervals.
- Retention-acquisition boundary.
- External/latent/parametric crossover.
- Resource-failure and feasibility report.
- Raw-to-claim artifact graph.

12.4 4단계: dense scaling과 capacity

At most two methods로 axis별 eight fit points와 two held-out scales를 측정한다. Association load, cycle, bytes, live token, latent/adaptor bits, sleep FLOPs, queue load를 한 번에 다 바꾸지 않는다. Opaque payload Fano track, external coverage track, module routing track을 분리한다.

산출물.

- Threshold-anchored capacity knees 또는 KNEE_UNIDENTIFIED.
- A100-regime four-axis fit과 holdout.
- Conditional bits/parameter와 complete-state efficiency.
- Coverage geometry-quantization result.
- Replay, refresh, rewrite lifetime-work curves.

12.5 5단계: systems design-space exploration

Shared, time-partitioned, separate, shard-local fleet를 trace-driven simulation과 A100 measurement로 비교한다. Simulation과 measured hardware를 분리한다. Queue, temporal clocks, local/global maintenance, adapter cache, data movement, model duplication을 함께 센다.

산출물.

- Wake P99와 sleep backlog frontier.
- Bytes moved와 physical residency.
- Queue-before-storage 여부.
- Fleet and locality crossover.
- Delete burst와 poison fault recovery.

12.6 6단계: external validity

LoCoMo, LongMemEval, MemoryAgentBench, HaluMem, procedural/embodied track을 붙이되 controlled claim을 대체하지 않는다. Public benchmark는 real language validity를 주지만 causal operator isolation과 lifetime cap은 별도로 유지한다.

12.7 논문이 최소한 성립하는 조건

Grand universal law가 없어도 다음이면 강한 paper가 된다.

1. Reproducible lifetime stream.
2. Matched external, latent, user-parametric, hybrid destinations.
3. 적어도 하나의 held-out predictive crossover, capacity knee, 또는 informative null.
4. Retention과 acquisition의 분리.
5. Complete state와 lifecycle cost.
6. Negative result path.
7. Source에서 figure와 sentence까지 실행 가능한 provenance.
8. Biological deployment extrapolation의 명시적 제한.

Hybrid가 실패해도 benchmark와 null은 남는다. Sleep이라는 이름을 쓴 여러 시스템을 동일한 information/resource/governance boundary에서 비교하는 것 자체가 지식 진전이다. 다만 이 통합의 novelty나 priority는 별도의 인접문헌 claim chart 없이는 주장하지 않는다.

13. 결론: 모델이 “자는” 것이 아니라 기억 생애주기가 생긴다

[확립된 근거] Sleep-time compute는 2025년에 무에서 등장한 아이디어가 아니다. Replay, pseudorehearsal, fast/slow system, fixed-cadence consolidation, distillation, sparse isolation, generated data, learned offline action은 오래된 계보를 가진다. 2025–2026년에는 LLM과 agent system의 token-space context, external semantic memory, latent state, adapter, slow expert, memory harness라는 서로 다른 문헌 갈래가 공통의 post-deployment lifecycle 설계공간 후보로 함께 읽힌다. 그러나 감사된 corpus에는 이들을 모두 하나의 system에서 통합·검증한 사례가 없다.

[통합 추론] 앞으로의 학습을 배포 전, wake, sleep으로 나누는 framing은 유효하다. 그러나 세 phase는 고립된 cluster가 아니다. Wake가 exact event와 fast state를 만들고, sleep이 candidate transformation을 수행하며, long-term fabric이 canonical·derived state를 tiering하고, control plane이 verify·publish·undo한다. 물리 infra는 load에 따라 공유 또는 분리되지만 logical ownership과 transaction boundary는 명확해야 한다.

[통합 추론] External versus parametric은 철학적 선택이 아니라 routing decision이다. External은 audit·correction·delete에 강하고 retrieval·growth 비용을 낸다. Parametric은 reuse amortization과 implicit generalization에 강하고 interference·routing·unlearning 비용을 낸다. Latent와 adapter는 중간 cache다. 가장 방어 가능한 default는 external-first, parameter-last이며, repeated validated reuse와 낮은 volatility가 compilation을 정당화할 때만 승격하는 것이다.

[확립된 근거] 유한한 system은 영원히 같은 fidelity로 열린 stream을 기억할 수 없다. 다만 먼저 실패하는 것은 parameter bit만이 아니다. Addressability, plasticity, governance, queue, catalog, HBM, recovery reserve가 physical storage보다 먼저 binding될 수 있다. “무한 memory” 대신 어느 pressure가 언제 binding되고 어떤 operator가 그것을 다른 자원으로 이동시키는지를 측정해야 한다.

[제안 가설] Sleep-time compute의 진짜 scaling law는 한 줄짜리 N^α 보다 **crossover atlas**일 가능성이 높다. Reuse가 compilation을 정당화하는 경계, staleness와 run cost가 cadence를 정하는 경계, queue가 store보다 먼저 차는 경계, external coverage와 quantization이 만나는 경계, module isolation과 router tax가 만나는 경계, recoverability가 0이 되기 전 intervention 경계다. 각 관계는 held-out prediction에 성공한 범위 안에서만 법칙이다.

마지막으로 이 연구가 지켜야 할 문장은 단순하다.

Sleep은 더 많은 compute를 쓰는 시간이 아니라, 경험을 어느 state로 옮길지 결정하고 그 이동을 검증·취소할 수 있게 만드는 생애주기다.

다음 단계

이 정의를 받아들이면 연구 대상도 바뀐다. 모델 하나의 accuracy가 아니라 scheduler, sleep-data builder, external/parametric consolidator, destination router, versioned memory fabric, promotion verifier, deletion graph, wake/sleep fleet를 함께 설계해야 한다. 그 통합 system이 matched lifetime benchmark에서 강한 baseline을 이기는지, 어디서 실패하는지, 어떤 비용으로 기억하는지를 정직하게 보여주는 것이 다음 단계다.